

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**  
**TRƯỜNG ĐẠI HỌC BÁCH KHOA**  
**KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO**

**ĐỒ ÁN KỸ THUẬT MÁY TÍNH**

**Xây dựng thuật toán tính toán tọa độ mục tiêu từ dữ liệu GPS  
góc ngắm và khoảng cách**

**NGÀNH: Kỹ Thuật Máy Tính**

**HỘI ĐỒNG: .....**

**GVHD: TS VÕ TUẤN BÌNH**

**TKHD: .....**

**—o0o—**

**SVTH1: Huỳnh Gia Qui (2112138)**

**SVTH2: Bùi Nguyễn Thành Luân (2111700)**

**SVTH3: Nguyễn Trung Hiếu (2113357 )**

**TP. HỒ CHÍ MINH, 01/2026**

## Lời cảm ơn

Trong quá trình thực hiện đồ án “*Xây dựng thuật toán tính toán tọa độ mục tiêu từ dữ liệu GPS, góc ngắm và khoảng cách*”, nhóm chúng em đã nhận được nhiều sự giúp đỡ quý báu từ các thầy cô, gia đình và bạn bè. Chúng em xin chân thành bày tỏ lòng biết ơn sâu sắc đến những người đã góp phần vào sự hoàn thành của đồ án này.

Trước hết, nhóm xin gửi lời cảm ơn sâu sắc tới **Thầy Võ Tuấn Bình** — người hướng dẫn chính đã tận tình chỉ bảo, góp ý quý giá và tạo điều kiện thuận lợi cho nhóm trong suốt quá trình thực hiện đề tài. Những gợi ý chuyên môn và sự hỗ trợ của thầy là nền tảng quan trọng giúp nhóm hoàn thiện báo cáo này.

Chúng em cũng xin cảm ơn **Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa TP.HCM** đã cung cấp môi trường học tập, tài liệu và cơ sở vật chất cần thiết. Xin cảm ơn các thầy cô trong khoa đã giảng dạy và truyền đạt kiến thức quý báu.

Đặc biệt, nhóm xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ về mặt tinh thần và vật chất để nhóm có thể hoàn thành đồ án. Những góp ý, trao đổi ý tưởng và sự giúp đỡ kịp thời của các bạn đã giúp nhóm khắc phục nhiều khó khăn trong quá trình thực hiện.

Mặc dù đã cố gắng hoàn thành tốt nhất, đồ án còn tồn tại những hạn chế. Nhóm rất mong nhận được ý kiến đóng góp từ quý thầy cô và bạn đọc để đồ án được hoàn thiện hơn.

# Mục lục

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Lời cảm ơn                                                                   | 1  |
| 1 Giới thiệu                                                                 | 1  |
| 2 Tổng quan Nghiên cứu Liên quan                                             | 2  |
| 2.1 Lịch sử Phát triển Hệ thống Định vị                                      | 2  |
| 2.1.1 Từ Định vị Truyền thống đến GPS                                        | 2  |
| 2.1.2 Các Hệ thống GNSS Toàn cầu                                             | 2  |
| 2.2 Các Phương pháp Tính toán Geodesic                                       | 2  |
| 2.2.1 Mô hình Cầu và Haversine Formula                                       | 2  |
| 2.2.2 Vincenty Formula và Ellipsoid WGS-84                                   | 3  |
| 2.2.3 Karney Algorithm                                                       | 3  |
| 2.3 Nghiên cứu về Sensor Fusion                                              | 3  |
| 2.3.1 Kalman Filter trong GPS/INS Integration                                | 3  |
| 2.3.2 Adaptive Filtering                                                     | 4  |
| 2.4 Hệ thống Web-based GIS và Mapping                                        | 4  |
| 2.4.1 Web Mapping Libraries                                                  | 4  |
| 2.4.2 Map Projections và Web Mercator                                        | 5  |
| 2.5 Ứng dụng Thực tế                                                         | 5  |
| 2.5.1 Military Applications                                                  | 5  |
| 2.5.2 Search and Rescue                                                      | 5  |
| 2.5.3 Surveying and Construction                                             | 5  |
| 2.6 Khoảng trống Nghiên cứu                                                  | 6  |
| 3 Cơ sở lý thuyết                                                            | 7  |
| 3.1 Hệ thống Định vị Toàn cầu GPS                                            | 7  |
| 3.1.1 GPS là gì                                                              | 7  |
| 3.1.2 Cấu trúc hệ thống                                                      | 7  |
| 3.1.3 Nguyên lý hoạt động                                                    | 7  |
| 3.1.4 Dữ liệu GPS                                                            | 7  |
| 3.2 Tam giác cầu                                                             | 8  |
| 3.2.1 Định nghĩa và tính chất                                                | 8  |
| 3.2.2 Mối Liên Hệ Lượng Giác Cầu Và Vector Định Hướng 3D                     | 8  |
| 3.3 Phép Chiếu Bản Đồ                                                        | 8  |
| 3.3.1 Khái niệm                                                              | 8  |
| 3.3.2 Các Phép Chiếu Bản Đồ Phổ Biến                                         | 8  |
| 3.4 Hệ Tọa Độ Không Gian                                                     | 9  |
| 3.4.1 Hệ tọa độ trắc địa Geodetic (Kinh độ, Vĩ độ, Độ cao)                   | 9  |
| 3.4.2 Hệ tọa độ địa tâm địa cố ECEF (Earth-Centered, Earth-Fixed)            | 9  |
| 3.4.3 Hệ tọa độ ENU (East-North-Up)                                          | 9  |
| 3.4.4 Hệ tọa độ NED (North-East-Down)                                        | 9  |
| 3.4.5 Hệ quy chiếu ellipsoid chuẩn WGS-84                                    | 10 |
| 3.5 Chuyển đổi giữa các hệ tọa độ                                            | 10 |
| 3.5.1 Chuyển đổi Geodetic ( $\varphi, \lambda, h$ ) sang ECEF ( $X, Y, Z$ )  | 10 |
| 3.5.2 Chuyển đổi ECEF $\leftrightarrow$ ENU                                  | 10 |
| 3.5.2.a Từ ECEF sang ENU                                                     | 10 |
| 3.5.2.b Từ ENU sang ECEF                                                     | 11 |
| 3.5.3 Chuyển đổi ECEF $\leftrightarrow$ NED                                  | 11 |
| 3.6 Góc Azimuth và Elevation                                                 | 11 |
| 3.6.1 Azimuth (Góc phương vị)                                                | 11 |
| 3.6.2 Elevation (Góc nâng)                                                   | 11 |
| 3.6.3 Chuyển Azimuth/Elevation Thành Vector Định Hướng Trong ENU/NED         | 11 |
| 3.6.3.a Vector định hướng trong ENU                                          | 11 |
| 3.6.3.b Vector định hướng trong NED                                          | 11 |
| 3.7 Xác định điểm mục tiêu từ vector định hướng và khoảng cách đo bằng laser | 12 |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Xử lý dữ liệu cảm biến GPS, IMU và Laser</b>                             | <b>13</b> |
| 4.1      | Giới thiệu và đặc tính từng loại cảm biến . . . . .                         | 13        |
| 4.1.1    | IMU (Inertial Measurement Unit) . . . . .                                   | 13        |
| 4.1.2    | Laser Rangefinder . . . . .                                                 | 13        |
| 4.2      | Các lỗi cảm biến và ảnh hưởng đến hệ thống định vị . . . . .                | 13        |
| 4.2.1    | Lỗi GPS . . . . .                                                           | 13        |
| 4.2.2    | Lỗi cảm biến IMU . . . . .                                                  | 14        |
| 4.2.3    | Lỗi laser rangefinder . . . . .                                             | 14        |
| 4.3      | Kỹ thuật hiệu chuẩn cảm biến thực tế . . . . .                              | 14        |
| 4.3.1    | Nguyên tắc hiệu chuẩn cảm biến . . . . .                                    | 14        |
| 4.3.1.a  | Hiệu chuẩn GPS . . . . .                                                    | 14        |
| 4.3.1.b  | Hiệu chuẩn IMU . . . . .                                                    | 14        |
| 4.3.1.c  | Hiệu chuẩn laser rangefinder . . . . .                                      | 15        |
| 4.3.2    | Một số phương pháp hiệu chuẩn tự động/online . . . . .                      | 15        |
| 4.4      | Pipeline xử lý dữ liệu cảm biến thực tế . . . . .                           | 15        |
| 4.4.1    | Cấu trúc pipeline tổng thể . . . . .                                        | 15        |
| 4.4.2    | Thực hiện sensor fusion GPS + IMU + Laser . . . . .                         | 15        |
| 4.5      | Thuật toán lọc nhiễu và hợp nhất dữ liệu cảm biến (Sensor Fusion) . . . . . | 16        |
| 4.5.1    | Extended Kalman Filter (EKF) . . . . .                                      | 16        |
| 4.5.2    | Adaptive Filter (Bộ lọc thích nghi) . . . . .                               | 16        |
| 4.6      | Kịch bản kiểm thử và xây dựng bộ dữ liệu thực/mô phỏng . . . . .            | 17        |
| 4.6.1    | Kịch bản kiểm thử thực nghiệm . . . . .                                     | 17        |
| 4.6.2    | Đánh giá hiệu năng thuật toán . . . . .                                     | 17        |
| <b>5</b> | <b>Web Map Use Cases</b>                                                    | <b>19</b> |
| 5.1      | Tổng quan về Web Map Viewer . . . . .                                       | 19        |
| 5.1.1    | Định nghĩa về Web Map Viewer . . . . .                                      | 19        |
| 5.1.2    | Mục tiêu của Web Map Viewer . . . . .                                       | 19        |
| 5.1.3    | Chức năng chính . . . . .                                                   | 19        |
| 5.1.4    | Lợi ích khi sử dụng Web Map Viewer . . . . .                                | 20        |
| 5.2      | Use Cases . . . . .                                                         | 20        |
| 5.2.1    | MILITARY & DEFENSE . . . . .                                                | 20        |
| 5.2.2    | Search & Rescue (SAR) . . . . .                                             | 21        |
| 5.2.3    | Tourism & Recreation . . . . .                                              | 22        |
| 5.2.4    | Education & Training . . . . .                                              | 23        |
| 5.3      | Technical Architecture . . . . .                                            | 24        |
| 5.3.1    | System Components . . . . .                                                 | 24        |
| 5.3.2    | Technology Stack . . . . .                                                  | 24        |
| 5.4      | Prototype phần mềm mô phỏng (Leaflet Web Map) . . . . .                     | 25        |
| 5.4.1    | Mục tiêu . . . . .                                                          | 25        |
| 5.4.2    | Công nghệ sử dụng . . . . .                                                 | 25        |
| 5.4.3    | Cấu trúc giao diện . . . . .                                                | 25        |
| 5.4.4    | Nguyên lý hoạt động . . . . .                                               | 26        |
| 5.4.5    | Kết quả mô phỏng . . . . .                                                  | 26        |
| 5.4.6    | Đánh giá . . . . .                                                          | 27        |
| <b>6</b> | <b>Yêu cầu về hệ thống</b>                                                  | <b>28</b> |
| 6.1      | Yêu cầu chức năng . . . . .                                                 | 28        |
| 6.1.1    | Nhập dữ liệu quan sát . . . . .                                             | 28        |
| 6.1.2    | Validation dữ liệu đầu vào . . . . .                                        | 28        |
| 6.1.3    | Tính toán tọa độ mục tiêu . . . . .                                         | 28        |
| 6.1.4    | Hiển thị kết quả trên bản đồ . . . . .                                      | 28        |
| 6.1.5    | Xuất và chia sẻ dữ liệu . . . . .                                           | 29        |
| 6.2      | Yêu cầu phi chức năng . . . . .                                             | 29        |
| 6.2.1    | Hiệu năng . . . . .                                                         | 29        |
| 6.2.2    | Khả năng sử dụng (Usability) . . . . .                                      | 29        |
| 6.2.3    | Tương thích . . . . .                                                       | 29        |
| 6.2.4    | Bảo trì và mở rộng . . . . .                                                | 29        |
| 6.2.5    | Độ tin cậy . . . . .                                                        | 30        |
| 6.2.6    | Bảo mật . . . . .                                                           | 30        |

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Đặc tả chi tiết Web hệ thống GPS Target Calculating System</b>            | <b>31</b> |
| 7.1       | Tổng quan                                                                    | 31        |
| 7.2       | Frontend (Giao diện người dùng)                                              | 31        |
| 7.3       | Backend (FastAPI)                                                            | 32        |
| 7.4       | Database (PostgreSQL + Alembic)                                              | 32        |
| 7.5       | Hosting Platform (Render)                                                    | 32        |
| 7.6       | Luồng dữ liệu trong hệ thống                                                 | 32        |
| <b>8</b>  | <b>Chi tiết Hiện thực và Phân tích Mã nguồn</b>                              | <b>33</b> |
| 8.1       | Kiến trúc Chi tiết                                                           | 33        |
| 8.1.1     | Mô hình MVC biến thể                                                         | 33        |
| 8.1.2     | Data Flow Architecture                                                       | 34        |
| 8.2       | Module coordinateCalculator.js - Phân tích Chi tiết                          | 34        |
| 8.2.1     | Constants và Configuration                                                   | 34        |
| 8.2.2     | Core Algorithm: Haversine Implementation                                     | 35        |
| 8.2.3     | Validation Strategy                                                          | 36        |
| 8.3       | Module mapView.js - Phân tích Chi tiết                                       | 36        |
| 8.3.1     | State Management                                                             | 36        |
| 8.3.2     | Dynamic Map Updates                                                          | 37        |
| 8.4       | Event Handling và User Interactions                                          | 38        |
| 8.4.1     | Event Delegation Pattern                                                     | 38        |
| 8.4.2     | Keyboard Shortcuts                                                           | 38        |
| 8.5       | Performance Optimization                                                     | 39        |
| 8.5.1     | Computation Performance                                                      | 39        |
| 8.5.2     | Memory Management                                                            | 39        |
| 8.5.3     | Network Optimization                                                         | 39        |
| 8.6       | Error Handling và Robustness                                                 | 39        |
| 8.6.1     | Error Types                                                                  | 39        |
| 8.6.2     | Graceful Degradation                                                         | 40        |
| 8.7       | Code Quality và Best Practices                                               | 40        |
| 8.7.1     | JSDoc Documentation                                                          | 40        |
| 8.7.2     | Naming Conventions                                                           | 40        |
| 8.7.3     | Code Metrics                                                                 | 40        |
| <b>9</b>  | <b>Lan truyền sai số từ ENU/ECEF sang đầu ra địa lý</b>                      | <b>41</b> |
| 9.1       | Các lỗi cảm biến và ảnh hưởng đến hệ thống định vị                           | 41        |
| 9.1.1     | Lỗi GPS                                                                      | 41        |
| 9.1.2     | Lỗi cảm biến IMU                                                             | 41        |
| 9.1.3     | Lỗi laser rangefinder                                                        | 41        |
| 9.2       | Công thức sai số (RSS)                                                       | 42        |
| 9.3       | Lan truyền Hiệp phương sai (Covariance Propagation) qua các Ma trận Jacobian | 42        |
| 9.3.1     | Độ lệch Chuẩn (Vô hướng) đến Ma trận Hiệp phương sai (Vector/Matrix)         | 42        |
| 9.3.2     | Luật Lan truyền Bất định (LPU) và Ma trận Jacobian                           | 43        |
| 9.3.3     | Lan truyền Sai số thông qua các Phép biến đổi Địa lý (Geodetic)              | 44        |
| 9.3.4     | So sánh Định lượng: RSS so với Lan truyền Hiệp phương sai (Jacobian)         | 45        |
| <b>10</b> | <b>Xem xét thuật toán tính toán tọa độ mục tiêu khác</b>                     | <b>46</b> |
| 10.1      | Tổng quan                                                                    | 46        |
| 10.2      | Ký hiệu và hằng số                                                           | 46        |
| 10.3      | Thuật toán Flat-Earth (cải tiến)                                             | 46        |
| 10.3.1    | Bước 1: Phân tách khoảng cách theo phương ngang và đứng                      | 46        |
| 10.3.2    | Bước 2: Tọa độ ENU của mục tiêu so với người quan sát                        | 46        |
| 10.3.3    | Bước 3: Bán kính cong tại vĩ độ trung bình $\phi_m$                          | 47        |
| 10.3.4    | Bước 4: Quy đổi từ ENU sang thay đổi tọa độ địa lý                           | 47        |
| 10.3.5    | Bước 5: Tọa độ mục tiêu                                                      | 47        |
| 10.4      | Ví dụ minh họa (chi tiết) — Flat-Earth (improved)                            | 48        |
| 10.5      | Kết quả thử nghiệm và so sánh                                                | 49        |
| 10.6      | Phân tích sai số và lựa chọn thuật toán                                      | 49        |
| 10.7      | Đánh giá và biểu diễn kết quả                                                | 50        |
| 10.8      | Kết luận                                                                     | 50        |

|           |                                        |           |
|-----------|----------------------------------------|-----------|
| <b>11</b> | <b>Tổng kết và Hướng phát triển</b>    | <b>51</b> |
| 11.1      | Tổng kết Giai đoạn 1 . . . . .         | 51        |
| 11.2      | Hạn chế . . . . .                      | 51        |
| 11.3      | Hướng phát triển Giai đoạn 2 . . . . . | 51        |

**Danh sách hình vẽ**

|   |                                                                            |    |
|---|----------------------------------------------------------------------------|----|
| 1 | Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet . . . . . | 25 |
| 2 | Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet . . . . . | 26 |
| 3 | Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet . . . . . | 27 |
| 4 | Kiến trúc prototype của Web . . . . .                                      | 31 |
| 5 | Luồng dữ liệu trong hệ thống . . . . .                                     | 34 |
| 6 | Đồ thị biểu diễn kết quả so sánh sai số giữa 2 mô hình . . . . .           | 50 |

# Danh sách bảng

|    |                                                                                                |    |
|----|------------------------------------------------------------------------------------------------|----|
| 1  | So sánh các hệ thống GNSS toàn cầu [7]                                                         | 2  |
| 2  | So sánh các thuật toán tính geodesic                                                           | 3  |
| 3  | Các tham số elipsoid WGS-84                                                                    | 10 |
| 4  | Các nguồn sai số trong tín hiệu GPS                                                            | 13 |
| 5  | Các loại lỗi điển hình trong hệ thống quán tính (gyro)                                         | 14 |
| 6  | Các yếu tố môi trường ảnh hưởng đến độ chính xác đo khoảng cách trong truyền sóng sub-millimet | 14 |
| 7  | So sánh EKF và Adaptive Filter                                                                 | 16 |
| 8  | Các chỉ số quan trọng để đánh giá hiệu năng thuật toán                                         | 17 |
| 9  | Performance benchmark các functions chính                                                      | 39 |
| 10 | Code quality metrics                                                                           | 40 |
| 11 | Các nguồn sai số trong tín hiệu GPS                                                            | 41 |
| 12 | Các loại lỗi điển hình trong hệ thống quán tính (gyro)                                         | 41 |
| 13 | Các yếu tố môi trường ảnh hưởng đến độ chính xác đo khoảng cách trong truyền sóng sub-millimet | 41 |
| 14 | So sánh kết quả giữa hai thuật toán                                                            | 49 |



## 1 Giới thiệu

- Trong các hệ thống chỉ huy - điều khiển hỏa lực, việc tính toán chính xác vị trí mục tiêu dựa trên thông tin đo thực địa là yếu tố quyết định thành công. Đồ án này nhằm xây dựng thuật toán xác định tọa độ địa lý (kinh độ, vĩ độ, cao độ) của mục tiêu từ các dữ liệu: vị trí GPS của người quan sát, góc ngắm (azimuth, elevation) đo từ IMU/la bàn, khoảng cách trực tiếp đo bằng laser rangefinder. Dữ liệu này sẽ được chuyển đổi thành nhiều hệ tọa độ trung gian (ECEF, ENU/NED) để thuận tiện cho xử lý số, góp phần hiển thị tọa độ mục tiêu trên bản đồ số (WebGIS), phục vụ nhiều ứng dụng quân sự và dân sự.
- Việc xác định tọa độ mục tiêu trong không gian thực dựa trên thông tin vị trí GPS, góc ngắm (azimuth, elevation) và khoảng cách đo bằng laser là bài toán nền tảng trong lĩnh vực quân sự (chỉ huy hỏa lực, định vị mục tiêu), GIS thực địa, robotics và tự động hóa di động. Hệ thống này có vai trò “giao tiếp” giữa thế giới thực (tọa độ địa lý) và bài toán động học, phục vụ cả việc tác chiến chính xác, khảo sát bản đồ cũng như định vị đối tượng di động hoặc cố định trong môi trường không đồng nhất.
- Yêu cầu của đồ án hướng đến việc thu thập, tổng hợp, xử lý các thông tin từ tổ hợp đa cảm biến (GPS, cảm biến đo khoảng cách laser LiDAR hoặc Time-of-Flight, la bàn điện tử...), tính toán kết hợp cả góc ngắm không gian (phổ biến là azimuth/elevation), đồng thời giải bài toán chuyển đổi qua lại giữa các hệ tọa độ bản đồ thực tế (WGS84, VN2000, UTM v.v...).

## 2 Tổng quan Nghiên cứu Liên quan

Phần này trình bày tổng quan về các nghiên cứu, hệ thống và phương pháp liên quan đến tính toán tọa độ mục tiêu từ dữ liệu GPS, góc phương vị và khoảng cách. Việc nghiên cứu các công trình trước đây giúp định vị đúng vị trí của đề tài, xác định khoảng trống nghiên cứu và đưa ra định hướng phát triển phù hợp.

### 2.1 Lịch sử Phát triển Hệ thống Định vị

#### 2.1.1 Từ Định vị Truyền thống đến GPS

Trước khi GPS ra đời, các phương pháp định vị chủ yếu dựa vào:

- **Thiên văn định vị:** Sử dụng quan sát các thiên thể (mặt trời, sao) để xác định vị trí. Phương pháp này được hàng hải sử dụng từ thế kỷ 15, có độ chính xác khoảng 1-2 hải lý (~2-4 km) [8].
- **Tam giác đạc:** Đo góc và khoảng cách đến các điểm mốc đã biết. Được Charles Mason và Jeremiah Dixon sử dụng để đo đạc biên giới Maryland-Pennsylvania (1763-1767), độ chính xác ~10m với khoảng cách ngắn.
- **Radio navigation:** LORAN (Long Range Navigation) phát triển trong Thế chiến II, sử dụng sự chênh lệch thời gian tín hiệu radio từ nhiều trạm phát. Độ chính xác ~200m-2km tùy khoảng cách [9].

GPS (Global Positioning System) được Bộ Quốc phòng Mỹ phát triển từ thập niên 1970, hoạt động chính thức năm 1995 với 24 vệ tinh. Đây là bước nhảy vọt về độ chính xác (từ km xuống m) và khả năng phủ sóng toàn cầu 24/7.

#### 2.1.2 Các Hệ thống GNSS Toàn cầu

Hiện nay có 4 hệ thống định vị vệ tinh toàn cầu (GNSS):

| Hệ thống | Quốc gia   | Số vệ tinh | Độ chính xác   | Ghi chú              |
|----------|------------|------------|----------------|----------------------|
| GPS      | Mỹ         | 31         | 5-10m          | Hoạt động từ 1995    |
| GLONASS  | Nga        | 24         | 5-10m          | Phục hồi đầy đủ 2011 |
| Galileo  | EU         | 30         | 1m (dân sự)    | Hoạt động từ 2016    |
| BeiDou   | Trung Quốc | 35         | 10m (toàn cầu) | Hoàn thành 2020      |

**Bảng 1:** So sánh các hệ thống GNSS toàn cầu [7]

Việc kết hợp nhiều hệ thống GNSS (multi-GNSS) giúp:

- Tăng số lượng vệ tinh khả dụng (từ 8-12 lên 20-30)
- Cải thiện độ chính xác (geometric dilution of precision - GDOP tốt hơn)
- Tăng độ tin cậy trong môi trường đô thị (urban canyon)

### 2.2 Các Phương pháp Tính toán Geodesic

#### 2.2.1 Mô hình Cầu và Haversine Formula

Công thức Haversine được R.W. Sinnott công bố năm 1984 [17], dựa trên giả định Trái Đất là hình cầu hoàn hảo với bán kính  $R = 6371$  km. Công thức này đơn giản nhưng hiệu quả cho khoảng cách ngắn:

$$\begin{aligned}a &= \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \\c &= 2 \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right) \\d &= R \cdot c\end{aligned}\tag{1}$$

**Ưu điểm:**

- Tính toán nhanh (~0.002ms)
- Code đơn giản, dễ implement
- Độ chính xác chấp nhận được (<0.5% với  $d < 100$ km)

#### Nhược điểm:

- Sai số tăng nhanh với khoảng cách lớn
- Không chính xác ở vĩ độ cao ( $>60^\circ$ )
- Bỏ qua hình dạng ellipsoid thực của Trái Đất

### 2.2.2 Vincenty Formula và Ellipsoid WGS-84

Thaddeus Vincenty (1975) phát triển công thức chính xác trên ellipsoid WGS-84 [21], sử dụng phương pháp lặp (iterative) để tìm geodesic (đường ngắn nhất trên ellipsoid).

Công thức Vincenty direct problem (tìm điểm đích từ điểm xuất phát, azimuth và khoảng cách):

$$\begin{aligned}\tan U_1 &= (1 - f) \tan \varphi_1 \\ \sigma_1 &= \arctan 2(\tan U_1, \cos \alpha_1) \\ \sin \alpha &= \cos U_1 \sin \alpha_1 \\ \cos^2 \alpha &= 1 - \sin^2 \alpha \\ u^2 &= \cos^2 \alpha \frac{a^2 - b^2}{b^2}\end{aligned}\tag{2}$$

Sau đó lặp để tìm  $\sigma$  (angular distance), và cuối cùng tính  $\varphi_2, \lambda_2$ .

#### Ưu điểm:

- Độ chính xác cực cao ( $\sim 0.1\text{mm}$  trên  $1000\text{km}$ )
- Stable với hầu hết các trường hợp
- Chuẩn được sử dụng trong surveying/geodesy

#### Nhược điểm:

- Phức tạp hơn  $\sim 10\text{x}$  so với Haversine
- Chậm hơn  $\sim 10\text{x}$  ( $\sim 0.02\text{ms}$ )
- Có thể không converge gần antipodal points

### 2.2.3 Karney Algorithm

Charles Karney (2013) cải tiến Vincenty với thuật toán ổn định hơn [10], giải quyết được vấn đề convergence và cung cấp độ chính xác  $\sim 15$  nanometers.

#### So sánh các thuật toán:

| Thuật toán | Độ chính xác               | Tốc độ     | Độ phức tạp | Stability |
|------------|----------------------------|------------|-------------|-----------|
| Haversine  | 0.5% $\rightarrow$ 100km   | Nhanh      | Thấp        | Tốt       |
| Vincenty   | 0.1mm $\rightarrow$ 1000km | Trung bình | Cao         | Khá tốt   |
| Karney     | 15nm $\rightarrow$ global  | Trung bình | Cao         | Rất tốt   |

**Bảng 2:** So sánh các thuật toán tính geodesic

## 2.3 Nghiên cứu về Sensor Fusion

### 2.3.1 Kalman Filter trong GPS/INS Integration

Kalman Filter (1960) là nền tảng của sensor fusion hiện đại. Extended Kalman Filter (EKF) được ứng dụng rộng rãi trong tích hợp GPS và IMU (Inertial Measurement Unit) [7].

#### Ý tưởng cốt lõi:

- GPS: Độ chính xác cao nhưng tần số thấp (1-10 Hz), có thể bị mất tín hiệu
- IMU: Tần số cao (100-1000 Hz), nhưng drift theo thời gian
- Kết hợp: Lấy ưu điểm của cả hai, giảm nhược điểm

Mô hình toán học EKF cho GPS/IMU:

$$\begin{aligned}\mathbf{x}_{k|k-1} &= f(\mathbf{x}_{k-1}, \mathbf{u}_k) \quad (\text{Prediction}) \\ \mathbf{P}_{k|k-1} &= \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k \\ \mathbf{K}_k &= \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \\ \mathbf{x}_k &= \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\mathbf{x}_{k|k-1})) \quad (\text{Update})\end{aligned} \quad (3)$$

Trong đó:

- $\mathbf{x}$ : State vector (position, velocity, orientation, IMU biases)
- $\mathbf{P}$ : Covariance matrix
- $\mathbf{K}$ : Kalman gain
- $\mathbf{Q}$ : Process noise covariance
- $\mathbf{R}$ : Measurement noise covariance

### 2.3.2 Adaptive Filtering

Trong môi trường động (GPS intermittent, varying noise), Adaptive Kalman Filter tự động điều chỉnh  $\mathbf{Q}$  và  $\mathbf{R}$  dựa trên innovation sequence [16]:

$$\begin{aligned}\mathbf{r}_k &= \mathbf{z}_k - h(\mathbf{x}_{k|k-1}) \quad (\text{Innovation}) \\ \mathbf{C}_r &= \frac{1}{N} \sum_{i=k-N+1}^k \mathbf{r}_i \mathbf{r}_i^T \quad (\text{Sample covariance}) \\ \hat{\mathbf{R}}_k &= \mathbf{C}_r - \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T \quad (\text{Adaptive R})\end{aligned} \quad (4)$$

## 2.4 Hệ thống Web-based GIS và Mapping

### 2.4.1 Web Mapping Libraries

Các thư viện web mapping phổ biến:

#### 1. Leaflet.js [1]:

- Open-source, lightweight (~42KB)
- Mobile-friendly
- Plugin ecosystem phong phú
- Sử dụng: OpenStreetMap, MapBox, custom tiles

#### 2. OpenLayers:

- More features (~250KB), phức tạp hơn
- Hỗ trợ nhiều projections (EPSG codes)
- Vector tiles, 3D capabilities

#### 3. Mapbox GL JS:

- GPU-accelerated rendering
- Beautiful styling, smooth interactions
- Commercial (cần API key)

#### 4. Google Maps API:

- Comprehensive features
- Best data coverage
- Expensive (7/1000 loads)

#### Lựa chọn Leaflet cho dự án:

- Open-source, không giới hạn
- Dễ nhẹ cho prototype/MVP
- Community support tốt
- Dễ offline (với MBTiles)

#### 2.4.2 Map Projections và Web Mercator

Hầu hết web maps sử dụng Web Mercator (EPSG:3857), một biến thể của Mercator projection [18]:

$$\begin{aligned}x &= R \cdot \lambda \\y &= R \cdot \ln \left[ \tan \left( \frac{\pi}{4} + \frac{\varphi}{2} \right) \right]\end{aligned}\tag{5}$$

#### Ưu điểm:

- Conformal (bảo toàn góc, hình dạng local)
- Rhumb lines (constant bearing) là đường thẳng
- Tiles dễ cache (256×256 pixels)

#### Nhược điểm:

- Biến dạng diện tích lớn (Greenland ~ Africa)
- Không dùng được cho vĩ độ >85°
- Không phải geodesic (đường ngắn nhất bị cong)

### 2.5 Ứng dụng Thực tế

#### 2.5.1 Military Applications

- **Artillery Targeting Systems:** Hệ thống như AFATDS (Advanced Field Artillery Tactical Data System) sử dụng GPS+IMU+laser rangefinder để tính toán fire missions [9].
- **Forward Observer Tools:** ATAK (Android Tactical Assault Kit) - Android app cho soldiers, tính toán target coordinates từ observer position + bearing + distance.
- **Precision-Guided Munitions:** GPS-guided bombs (JDAM) có độ chính xác ~5m CEP (Circular Error Probable).

#### 2.5.2 Search and Rescue

- SAR teams sử dụng handheld GPS + compass + visual range estimation
- RECCO system: Reflector-based location (không cần battery)
- Drone-assisted SAR: Tích hợp gimbal angles để tính victim location

#### 2.5.3 Surveying and Construction

- Total Stations: Tích hợp theodolite + EDM (Electronic Distance Measurement), độ chính xác ~1mm -> 1km
- RTK GPS: Real-Time Kinematic, cm-level accuracy
- BIM (Building Information Modeling): Tích hợp GPS data vào 3D models

## 2.6 Khoảng trống Nghiên cứu

Từ tổng quan nghiên cứu, nhận thấy:

1. **Thiếu hệ thống lightweight:** Các giải pháp thương mại (ATAK, AFATDS) phức tạp, đắt tiền. Cần giải pháp đơn giản, open-source cho education/training.
2. **Thiếu phân tích sai số tổng hợp:** Hầu hết nghiên cứu tập trung vào GPS error hoặc IMU error riêng lẻ, ít phân tích error propagation toàn hệ thống.
3. **Web-based gap:** Các công cụ hiện có chủ yếu là desktop/mobile apps, ít giải pháp web-based dễ triển khai.
4. **Thiếu so sánh Haversine vs Vincenty:** Ít nghiên cứu định lượng trade-off accuracy vs. performance cho use cases khác nhau.

Dự án này đóng góp vào các khoảng trống trên bằng cách:

- Xây dựng hệ thống open-source, web-based, lightweight
- Phân tích chi tiết error propagation từ sensors đến output
- So sánh định lượng Haversine vs. Vincenty
- Cung cấp educational tool với visualization trực quan

### 3 Cơ sở lý thuyết

#### 3.1 Hệ thống Định vị Toàn cầu GPS

##### 3.1.1 GPS là gì

GPS là hệ thống định vị toàn cầu dựa trên mạng lưới các vệ tinh nhân tạo quay quanh Trái Đất. Hệ thống này được thiết kế, xây dựng và vận hành bởi Bộ Quốc phòng Hoa Kỳ, ban đầu phục vụ mục đích quân sự, sau đó mở rộng cho dân sự từ thập niên 1980.

##### 3.1.2 Cấu trúc hệ thống

GPS gồm ba thành phần chính:

- Phần không gian (Space Segment): gồm 30 vệ tinh (27 vệ tinh hoạt động và 3 vệ tinh dự phòng) nằm trên các quỹ đạo xoay quanh Trái Đất. Chúng cách mặt đất 20.200 km, bán kính quỹ đạo 26.600 km. Chúng chuyển động ổn định và quay hai vòng quỹ đạo trong khoảng thời gian gần 24 giờ với vận tốc 7 nghìn dặm một giờ. Các vệ tinh trên quỹ đạo được bố trí sao cho các máy thu GPS trên mặt đất có thể nhìn thấy tối thiểu 4 vệ tinh vào bất kỳ thời điểm nào.
- Phần điều khiển (Control Segment): kiểm soát vệ tinh đi đúng hướng theo quỹ đạo và thông tin thời gian chính xác. Có 5 trạm kiểm soát đặt rải rác trên Trái Đất. Bốn trạm kiểm soát hoạt động một cách tự động, và một trạm kiểm soát là trung tâm. Bốn trạm này nhận tín hiệu liên tục từ những vệ tinh và gửi các thông tin này đến trạm kiểm soát trung tâm. Tại trạm kiểm soát trung tâm, nó sẽ sửa lại dữ liệu cho đúng và kết hợp với hai anten khác để gửi lại thông tin cho các vệ tinh. Ngoài ra, còn một trạm kiểm soát trung tâm dự phòng và sáu trạm quan sát chuyên biệt.
- Phần người dùng (User Segment): các thiết bị thu GPS (điện thoại, ô tô, tàu thuyền, máy bay...).

##### 3.1.3 Nguyên lý hoạt động

- Mỗi vệ tinh GPS phát tín hiệu chứa thời gian phát và vị trí chính xác  $(x_i, y_i, z_i)$  của vệ tinh tại thời điểm đó.
- Thiết bị thu GPS nhận tín hiệu và so sánh thời gian phát với thời gian nhận  $\rightarrow$  tính được khoảng cách đến vệ tinh  $R_i$ .
- Để xác định vị trí trong không gian 3D, cần ít nhất 4 vệ tinh:
  - 3 vệ tinh để tính vĩ độ, kinh độ, độ cao.
  - 1 vệ tinh bổ sung để hiệu chỉnh sai số đồng hồ trong thiết bị thu.
- Giải hệ phương trình 4 ẩn sau, ta được  $(X, Y, Z)$  là tọa độ thiết bị thu. Với  $(x_i, y_i, z_i)$  là tọa độ các vệ tinh tương ứng,  $R_i$  là khoảng cách từ vệ tinh tới thiết bị nhận,  $c$  là vận tốc ánh sáng trong chân không,  $t_i$  là thời gian phản hồi từ vệ tinh đến thiết bị nhận,  $\Delta t$  là sai số giữa vệ tinh và thiết bị nhận.

$$R_1 = \sqrt{(X - x_1)^2 + (Y - y_1)^2 + (Z - z_1)^2} = c(t_1 - t_0 \pm \Delta t) \quad (6)$$

$$R_2 = \sqrt{(X - x_2)^2 + (Y - y_2)^2 + (Z - z_2)^2} = c(t_2 - t_0 \pm \Delta t) \quad (7)$$

$$R_3 = \sqrt{(X - x_3)^2 + (Y - y_3)^2 + (Z - z_3)^2} = c(t_3 - t_0 \pm \Delta t) \quad (8)$$

$$R_4 = \sqrt{(X - x_4)^2 + (Y - y_4)^2 + (Z - z_4)^2} = c(t_4 - t_0 \pm \Delta t) \quad (9)$$

##### 3.1.4 Dữ liệu GPS

Các thiết bị GPS hiện đại xuất ra dữ liệu định vị toàn cầu dựa vào quan sát tối thiểu 4 vệ tinh để xác định vị trí 3D (lat, lon, alt) với độ trễ thấp và độ chính xác phụ thuộc số lượng vệ tinh hữu ích, điều kiện môi trường, model thiết bị thu.

- Định dạng truyền dẫn tiêu chuẩn: NMEA-0183, RINEX, RTCM... Trong hệ nhúng/robotics phổ biến nhất là dòng NMEA (GGA, RMC...) cung cấp nhanh các tham số tọa độ, chất lượng tín hiệu, số vệ tinh, độ chính xác đo lường, vận tốc v.v...
- Mã hóa thông điệp: giá trị vĩ độ, kinh độ (thập phân, DMS), độ cao (so với mực nước biển), thông số chất lượng (HDOP, VDOP), và tín hiệu thời gian chuẩn UTC.

**Các lỗi phổ biến khi lấy dữ liệu GPS:** che chắn/vật cản, nhiễu đa đường, nhiễu sóng, sai số đồng bộ thời gian, sai số khí quyển (điện ly, đối lưu), bị spoofing/jamming tấn công.

## 3.2 Tam giác cầu

### 3.2.1 Định nghĩa và tính chất

Tam giác cầu là hình được tạo thành bởi ba cung tròn lớn cắt nhau trên mặt cầu. Mỗi cạnh là một cung tròn lớn, còn các góc là giao của hai cung đó. Độ dài của cạnh không phải đo bằng độ dài (đơn vị mét) mà đo bằng góc ở tâm (tính bằng độ hoặc radian), bằng cung tròn nối hai điểm trên mặt cầu chia cho bán kính. Xuất hiện trong nhiều lĩnh vực thực tế: hàng hải, hàng không, thiên văn, đo đạc bản đồ, định hướng vệ tinh, quân sự...

### 3.2.2 Môi Liên Hệ Lượng Giác Cầu Và Vector Định Hướng 3D

Phép toán hình học giữa các điểm trên cầu có thể mô tả bằng tích vô hướng, tích có hướng, chuyển đổi tọa độ từ kinh vĩ độ sang vector 3D (n-vector):

- Với điểm trên cầu có vĩ độ  $\phi$ , kinh độ  $\lambda$ :

$$\begin{aligned}x &= R \cos \phi \cos \lambda \\y &= R \cos \phi \sin \lambda \\z &= R \sin \phi\end{aligned}$$

- Đây là nền tảng chuyển đổi dữ liệu địa lý sang không gian vector 3D, phục vụ cho các thuật toán dẫn đường, định hướng, hoặc mô phỏng quỹ đạo trên cầu. Từ vector 3D suy trở lại kinh vĩ độ.

## 3.3 Phép Chiếu Bản Đồ

### 3.3.1 Khái niệm

Phép chiếu bản đồ là phép toán học chuyển bề mặt cong 3D của Trái đất (hình cầu, ellipsoid) lên mặt phẳng bản đồ 2D. Không một phép chiếu nào bảo toàn toàn bộ hình dạng, diện tích, góc và khoảng cách. Các phép chiếu được tạo ra để tối ưu hóa một hoặc một vài thuộc tính tùy mục đích sử dụng.

### 3.3.2 Các Phép Chiếu Bản Đồ Phổ Biến

- Phép Chiếu Hình Trụ (Cylindrical Projection):** Chiếu điểm cầu lên hình trụ "bao" quanh quả cầu, sau đó trải phẳng.
  - Ưu điểm: Giữ vững góc tại xích đạo, thích hợp cho khu vực quanh xích đạo. Đường kinh tuyến và vĩ tuyến đều là đường thẳng song song, vuông góc nhau, dễ biểu diễn, thuận tiện cho đo đạc các vị trí gần đường tiếp xúc.
  - Nhược điểm: Biến dạng tăng dần về phía cực, diện tích các khu vực gần cực bị phóng đại mạnh. Không áp dụng tốt với lãnh thổ trải dài Bắc - Nam hoặc khu vực cực.
- Phép Chiếu Mercator (Mercator Projection):** Là hình trụ đứng đồng góc, công thức ( $\lambda$  là kinh độ,  $\phi$  là vĩ độ,  $\lambda_0$  là kinh tuyến chuẩn):

$$x = R(\lambda - \lambda_0) \quad y = R \ln(\tan(\pi/4 + \phi/2))$$

- Ưu điểm: Bảo toàn góc (hữu ích cho định hướng, hướng đi hàng hải, tuyến đường thẳng trên bản đồ là tuyến đường không đổi hướng la bàn). Dễ dàng chuyển đổi số liệu, tích hợp vào bản đồ WebGIS toàn cầu (Google Maps, OSM, Bing Maps đều dùng biến thể Mercator).
  - Nhược điểm: Biến dạng diện tích tăng rất mạnh về phía cực (ví dụ Greenland gần như lớn bằng châu Phi trên bản đồ Mercator); chỉ phù hợp thực tế cho vùng gần xích đạo.
- Phép Chiếu UTM (Universal Transverse Mercator)**
    - Là phép chiếu hình trụ ngang đồng góc, chia toàn cầu thành 60 múi, mỗi múi rộng 6 độ kinh độ (với số hiệu từ 1-60, bắt đầu từ 180° Tây).
    - Sử dụng trục hình trụ cắt cầu tại hai đường cong gần múi trung tâm (giảm biến dạng, phân bố đều sai số).



- Kinh tuyến giữa có hệ số biến dạng  $k < 1$  (thường  $k = 0.9996$ ).
- Biến dạng nhỏ, đều; phù hợp cho bản đồ tỷ lệ lớn, khu vực có diện tích lớn, lãnh thổ trải dài đông-tây/north-south. Là tiêu chuẩn toàn cầu trong trắc địa, quân sự, GPS, QGIS, hệ thống VN2000.
- Nhược điểm: Đòi hỏi thay đổi múi chiếu khi vượt biệt khu vực rộng, cần căn chỉnh số hiệu múi cho phù hợp từng khu vực.

### 3.4 Hệ Tọa Độ Không Gian

#### 3.4.1 Hệ tọa độ trắc địa Geodetic (Kinh độ, Vĩ độ, Độ cao)

Hệ tọa độ Geodetic (tọa độ trắc địa) mô tả vị trí một điểm trên bề mặt hoặc gần bề mặt Trái Đất bằng 3 tham số:

- Vĩ độ: Góc lệch giữa mặt phẳng xích đạo và phương đi qua tâm trái đất đến điểm cần xác định. Giá trị từ  $-90^\circ$  (cực Nam) đến  $+90^\circ$  (cực Bắc).
- Kinh độ: Góc giữa kinh tuyến đi qua điểm cần xác định và kinh tuyến gốc tại Greenwich, chạy từ  $-180^\circ$  tới  $180^\circ$ .
- Độ cao: Khoảng cách vuông góc từ điểm cần xác định tới mặt elip tham chiếu (ellipsoid), đơn vị thường là mét.

#### 3.4.2 Hệ tọa độ địa tâm địa cố ECEF (Earth-Centered, Earth-Fixed)

Hệ tọa độ ECEF là hệ tọa độ Đề-các (vuông góc) ba chiều, có:

- Gốc tọa độ: Tâm Trái Đất.
- Trục X: Đi qua giao điểm giữa mặt phẳng xích đạo và kinh tuyến gốc Greenwich (vĩ độ  $0^\circ$ , kinh độ  $0^\circ$ ).
- Trục Y: Nằm trong mặt phẳng xích đạo, cách trục X  $90^\circ$  về phía đông (vĩ độ  $0^\circ$ , kinh độ  $90^\circ$ ).
- Trục Z: Trùng với trục quay của Trái Đất, hướng về phía cực Bắc (vĩ độ  $90^\circ$ ).

**Đặc điểm:** ECEF là hệ tọa độ tuyệt đối, xoay đồng bộ với Trái Đất. Đơn vị là mét. Các điểm có tọa độ (X, Y, Z) trong ECEF có thể là điểm trên mặt đất, trong khí quyển, quỹ đạo vệ tinh hoặc ngoài không gian lân cận Trái Đất.

#### 3.4.3 Hệ tọa độ ENU (East-North-Up)

ENU là hệ tọa độ địa phương (Local Tangent Plane Coordinates), với:

- Gốc tọa độ đặt tại một điểm tham chiếu (người quan sát).
- Trục X - East: Hướng về phía Đông địa phương.
- Trục Y - North: Hướng về phía Bắc địa phương.
- Trục Z - Up: Hướng thẳng đứng lên trên (vuông góc với mặt ellipsoid tại điểm gốc).

Hệ ENU thuận tay phải, rất phù hợp cho các thuật toán điều hướng, robot, dẫn đường UAV, phân tích radar hoặc quy hoạch đô thị, khi các tính toán chỉ cần tập trung vào phạm vi một vùng nhỏ xung quanh người quan sát.

#### 3.4.4 Hệ tọa độ NED (North-East-Down)

NED là hệ tọa độ định hướng cục bộ nổi bật trong ngành hàng không, tàu biển, mô phỏng quán tính:

- Trục X - North: Hướng về phía Bắc địa phương.
- Trục Y - East: Hướng về phía Đông địa phương.
- Trục Z - Down: Hướng thẳng đứng xuống dưới (vuông góc với mặt ellipsoid, phía tâm Trái đất).

Hai hệ ENU và NED chỉ khác nhau thứ tự trục: NED có Z hướng xuống (“down” dương), phù hợp với một số ứng dụng động lực học tàu bay, tàu biển - các vật thể chủ yếu di chuyển/quan sát phía dưới máy bay.

### 3.4.5 Hệ quy chiếu ellipsoid chuẩn WGS-84

Để đảm bảo các phép chuyển đổi thông suốt và nhất quán trên toàn cầu, WGS-84 (World Geodetic System 1984) là ellipsoid tham chiếu thông nhất cho GPS - với các tham số:

| Tham số      | Ký hiệu | Giá trị               | Đơn vị |
|--------------|---------|-----------------------|--------|
| Bán trục lớn | $a$     | 6,378,137.0           | m      |
| Độ dẹt       | $f$     | $1/298.257223563$     | -      |
| Độ lệch tâm  | $e$     | 0.0818191908426       | -      |
| $e^2$        | $e^2$   | 0.0066943799901377997 | -      |
| Bán trục nhỏ | $b$     | 6,356,752.3142        | m      |

**Bảng 3:** Các tham số ellipsoid WGS-84

WGS-84 là mặt ellipsoid sát nhất mô phỏng hình dạng Trái Đất, được duy trì thường xuyên để thích hợp với di động kiến tạo mảng, mực nước biển và biến động trọng lực toàn cầu.

## 3.5 Chuyển đổi giữa các hệ tọa độ

### 3.5.1 Chuyển đổi Geodetic $(\varphi, \lambda, h)$ sang ECEF $(X, Y, Z)$

- $\varphi$ : vĩ độ tính bằng radian
- $\lambda$ : kinh độ tính bằng radian
- $h$ : cao độ so với ellipsoid
- $a$ : bán trục lớn của ellipsoid
- $e$ : độ lệch tâm của ellipsoid

1. Tính  $N$  (bán kính cong đơn vị kinh tuyến chính):

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}$$

2. Tọa độ ECEF:

$$\begin{aligned} X &= (N + h) \cdot \cos \varphi \cdot \cos \lambda \\ Y &= (N + h) \cdot \cos \varphi \cdot \sin \lambda \\ Z &= [N \cdot (1 - e^2) + h] \cdot \sin \varphi \end{aligned}$$

### 3.5.2 Chuyển đổi ECEF $\leftrightarrow$ ENU

#### 3.5.2.a Từ ECEF sang ENU

- Giả sử điểm gốc *local origin* có tọa độ địa lý  $(\phi_0, \lambda_0, h_0)$  — tính được tọa độ ECEF:  $(X_0, Y_0, Z_0)$ .
- Tính vector sai khác:

$$\Delta = \begin{bmatrix} dX \\ dY \\ dZ \end{bmatrix} = \begin{bmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{bmatrix}$$

- Xoay  $\Delta$  bằng ma trận quay  $R$  để đưa về hệ trục ENU:

$$\begin{bmatrix} E \\ N \\ U \end{bmatrix} = \begin{bmatrix} -\sin \lambda_0 & \cos \lambda_0 & 0 \\ -\sin \phi_0 \cos \lambda_0 & -\sin \phi_0 \sin \lambda_0 & \cos \phi_0 \\ \cos \phi_0 \cos \lambda_0 & \cos \phi_0 \sin \lambda_0 & \sin \phi_0 \end{bmatrix} \begin{bmatrix} dX \\ dY \\ dZ \end{bmatrix}$$

- Kết quả là tọa độ  $(E, N, U)$  trong hệ ENU tại gốc  $(\phi_0, \lambda_0, h_0)$ .

### 3.5.2.b Từ ENU sang ECEF

Đảo ngược phép quay và chuyển vị trí, tức là áp dụng ma trận chuyển đổi nghịch đảo (hoặc chuyển vị) để đưa tọa độ ENU về lại hệ ECEF.

### 3.5.3 Chuyển đổi ECEF $\leftrightarrow$ NED

Hai hệ ENU và NED chỉ khác thứ tự trục và một số dấu:

$$[N, E, D] = [E, N, U] \quad \text{với } D = -U$$

Ma trận quay cho NED về ECEF cũng có dạng gần giống ENU, chỉ đảo trục dọc trục đứng.

## 3.6 Góc Azimuth và Elevation

### 3.6.1 Azimuth (Góc phương vị)

Azimuth là góc nằm ngang trên mặt phẳng XY, xác định hướng nhìn hay hướng ngắm mục tiêu so với hướng chuẩn (thường là hướng Bắc địa lý). Trong các hệ ENU/NED, azimuth được quy ước như sau:

- Azimuth =  $0^\circ$  tương ứng hướng Bắc.
- Được đo theo chiều kim đồng hồ từ Bắc - tức là hướng Đông là  $90^\circ$ , Nam là  $180^\circ$ , Tây là  $270^\circ$ .

### 3.6.2 Elevation (Góc nâng)

Elevation là góc giữa vector quan sát với mặt phẳng ngang (XY) tại vị trí người quan sát:

- Góc elevation =  $0^\circ$ : hướng song song mặt đất (horizon).
- Góc elevation  $> 0^\circ$ : hướng lên trên đường chân trời.
- Góc elevation  $< 0^\circ$ : hướng xuống dưới mặt phẳng chân trời.

**Ký hiệu:** elevation thường dùng đơn vị độ ( $^\circ$ ), hoặc radian.

### 3.6.3 Chuyển Azimuth/Elevation Thành Vector Định Hướng Trong ENU/NED

#### 3.6.3.a Vector định hướng trong ENU

Giả sử bạn nhận được các giá trị azimuth (A) và elevation (E) ở độ, vector hướng trong hệ ENU sẽ là:

$$\begin{aligned}x_{ENU} &= \cos(E) \cdot \sin(A) \\y_{ENU} &= \cos(E) \cdot \cos(A) \\z_{ENU} &= \sin(E)\end{aligned}$$

**Quy ước quan trọng:**

- Azimuth A:  $0^\circ$  hướng Bắc (Y+),  $90^\circ$  hướng Đông (X+).
- Elevation E:  $0^\circ$  nằm ngang,  $+90^\circ$  thẳng đứng lên (Z+).

#### 3.6.3.b Vector định hướng trong NED

Khác biệt lớn nhất ở hệ NED là trục Z hướng xuống:

$$\begin{aligned}x_{NED} &= \cos(E) \cdot \cos(A) \\y_{NED} &= \cos(E) \cdot \sin(A) \\z_{NED} &= -\sin(E)\end{aligned}$$

**Ghi chú quy ước:**

- Azimuth vẫn tính như ENU, nhưng trục đặt lại.
- Độ dương/âm của thành phần z thay đổi để đảm bảo Z hướng xuống.

### 3.7 Xác định điểm mục tiêu từ vector định hướng và khoảng cách đo bằng laser

- Lấy vị trí người quan sát làm gốc hệ ENU.
  - Trong không gian 3D, để đơn giản hóa các phép tính hướng và vị trí, thường quy ước đặt gốc hệ ENU trùng với vị trí người quan sát trên mặt đất. Trục Ox (East) hướng về phía Đông, Oy (North) hướng về phía Bắc, Oz (Up) vuông góc mặt đất và hướng lên trời.
  - Khi gốc tọa độ được cố định tại người quan sát, bất kỳ điểm nào trong không gian gần gốc (phạm vi vài km) sẽ được biểu diễn nhờ ba giá trị ENU (E: Đông, N: Bắc, U: Cao). Điều này giúp phép cộng, trừ vector và chuyển đổi hệ để thực hiện hơn.
  - Ví dụ: Nếu một trạm đo đặt tại (lat0, lon0, h0), là gốc ENU, điểm mục tiêu sẽ tính toán tương đối với gốc này.
- Tính vector định hướng 3D từ azimuth, elevation (như phần 2.6.3)
- Xác định tọa độ mục tiêu trong ENU bằng phép cộng vector
  - Sau khi đã có vector định hướng 3D đơn vị V, tọa độ mục tiêu trong ENU sẽ được tính bằng phép cộng vị trí người quan sát với khoảng cách đo được nhân vector định hướng:

$$P_{target\_ENU} = P_{observer\_ENU} + D.V$$

**Trong đó:**

- $P_{target\_ENU} = (0,0,0)$  do lấy người quan sát là gốc
- D là khoảng cách đo bằng laser (đơn vị mét)
- V là vectơ định hướng 3D đơn vị đã tính
- Chuyển ENU sang ECEF Công thức ma trận chuyển đổi:  $P_{ECEF} = R_{ENU \rightarrow ECEF} . P_{ENU} + Observer_{ECEF}$   
Trong đó:
  - $\lambda$  là kinh độ,  $\phi$  là vĩ độ,  $Observer_{ECEF}$  được tính theo phần 2.5.1
- Chuyển ECEF sang địa lý (kinh vĩ độ, cao độ) Từ điểm ECEF (X, Y, Z), chuyển về hệ tọa độ địa lý dùng công thức chuẩn WGS84( $\lambda, \phi, h$ ):

$$\lambda = \arctan 2(Y, X)$$

$$p = \sqrt{X^2 + Y^2}$$

$$\phi = \arctan \left( \frac{Z}{p} \cdot \left( 1 - e^2 \cdot \frac{a}{\sqrt{p^2 + (1 - e^2)Z^2}} \right)^{-1} \right)$$

$$h = \frac{p}{\cos \phi} - N(\phi)$$

$$N(\phi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}}$$

- Với các thông số xem lại trong bảng 1

## 4 Xử lý dữ liệu cảm biến GPS, IMU và Laser

### 4.1 Giới thiệu và đặc tính từng loại cảm biến

#### 4.1.1 IMU (Inertial Measurement Unit)

IMU là tổ hợp nhiều cảm biến: accelerometer (gia tốc kế), gyroscope (con quay hồi chuyển), magnetometer (la bàn số) cung cấp thông tin về gia tốc tuyến tính, vận tốc góc, phương hướng và trường từ trường của môi trường.

- Accelerometer: đo gia tốc (bao gồm cả trọng lực), dùng xác định phương trọng lực, đo rung động.
- Gyroscope: đo vận tốc quay, giúp xác định thay đổi hướng, tính toán góc nghiêng, phục vụ tính toán quán tính.
- Magnetometer: đo trường từ, dùng định hướng Bắc-Tây-Nam, hỗ trợ hiệu chuẩn phương vị.
- **Thông số cơ bản:**
  - Dải đo **accelerometer**:  $\pm 2g \dots \pm 16g$ , noise density đến  $0.01 - 0.02 m/s^2 / \sqrt{Hz}$ .
  - Dải đo **gyroscope**: lên tới  $\pm 2000^\circ/s$ , noise density tầm  $0.1 - 0.15^\circ/s / \sqrt{Hz}$ ; bias thường là vài  $^\circ/h$  (tốt đến vài trăm  $^\circ/h$  (rẻ)).
  - **Magnetometer**: độ nhạy tới  $\sim 1 \mu T$  (microtesla), noise  $< 0.5 \mu T / \sqrt{Hz}$ .

IMU hiện diện rộng rãi trong robot di động, máy bay không người lái UAV, xe hơi tự hành, máy đo chuyên dụng phục vụ đo đạc bản đồ, dò tìm dị thường địa vật lý.

#### 4.1.2 Laser Rangefinder

Laser Rangefinder là thiết bị đo khoảng cách chính xác dựa trên nguyên lý lan truyền của xung laser, hoạt động dựa trên time-of-flight, phase-shift hoặc triangulation.

- **Nguyên lý cơ bản:**
  - Time-of-flight: bắn xung laser  $\rightarrow$  đo thời gian tới-lui  $\rightarrow$  tính khoảng cách.
  - Phase-shift: đo độ lệch pha giữa sóng phát và sóng phản xạ, cho độ chính xác cao ở tầm gần và trung bình.
  - Triangulation: dùng định vị hình học điểm chiếu, chủ yếu cho đo dịch chuyển nhỏ, độ phân giải cao.
- **Độ chính xác:** Tùy cấu hình, có thể xuống tới 0.1mm (triangulation), cấp mm-cm (time-of-flight), sub-mm (phase-shift). Khoảng đo tới vài chục mét (nhỏ, dùng trong công nghiệp), tới vài km (quân sự).
- **Yếu tố ảnh hưởng:** Độ ẩm, bụi, mưa/động học khí quyển, mục tiêu hoặc vật cản, phân kỳ tia, hiệu ứng môi trường làm suy hao tín hiệu, độ phản xạ bề mặt mục tiêu.

### 4.2 Các lỗi cảm biến và ảnh hưởng đến hệ thống định vị

#### 4.2.1 Lỗi GPS

| Loại lỗi               | Mô tả                                                                 | Độ lớn (m)   |
|------------------------|-----------------------------------------------------------------------|--------------|
| Ionospheric delay      | Sai số do tín hiệu đi qua tầng ion, phụ thuộc tần số, khắc phục L1/L2 | $\pm 5-50$   |
| Tropospheric delay     | Sai số do hơi ẩm, áp suất khí quyển                                   | $\pm 0.5-30$ |
| Multipath distortion   | Phản xạ đa đường (địa hình, mặt nước, nhà cửa)                        | $\pm 1-150$  |
| Ephemeris errors       | Quỹ đạo vệ tinh bị lỗi thời gian, sai vị trí vệ tinh                  | $\pm 2-5$    |
| Satellite clock drift  | Trôi đồng hồ nguyên tử trên vệ tinh                                   | $\pm 0-1.5$  |
| Selective Availability | Sai số cố ý (đã tắt năm 2000)                                         | $\sim 100$   |
| Natural/Artificial EMI | Gây sai lệch/tắt tín hiệu (bão từ, jammer)                            | Variable     |

**Bảng 4:** Các nguồn sai số trong tín hiệu GPS

**Phân tích:** Các sai số trên cộng dồn thành tổng sai số định vị. Ở môi trường đô thị, hiệu ứng Multipath là nguyên nhân chính làm giảm đáng kể độ chính xác GPS.

#### 4.2.2 Lỗi cảm biến IMU

| Loại lỗi                               | Biểu hiện                                              | Độ lớn điển hình                                                                   |
|----------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------|
| Bias                                   | Lệch thông thường của gyro cả khi không có vận tốc góc | $0.1-1 \text{ }^\circ/\text{s}$ (accel), $0.01-0.1 \text{ }^\circ/\text{s}$ (gyro) |
| Scale factor                           | Sai số hệ số tỉ lệ giữa vận tốc góc và đầu ra          | $0.1-1\%$ (typical acc/gyro)                                                       |
| Random walk                            | Sự lệch do tín hiệu noise, trôi dần theo thời gian     | $0.01-0.1 \text{ }^\circ/\sqrt{\text{hr}}$                                         |
| Misalignment                           | Lệch trục cảm biến                                     | $< 0.1^\circ$                                                                      |
| Temperature drift                      | Lệch do nhiệt độ môi trường thay đổi                   | $0.1-1 \text{ mg}/^\circ\text{C}$ ; $0.1-1 \text{ }^\circ/\text{s}/^\circ\text{C}$ |
| Nonlinearity / Vibration rectification | Đáp ứng phi tuyến, ảnh hưởng rung                      | Không xác định                                                                     |

**Bảng 5:** Các loại lỗi điển hình trong hệ thống quán tính (gyro)

**Phân tích:** Lỗi *bias* và *random walk* của gyro làm drift lớn nhất cho hệ thống định vị quán tính; lỗi *scale factor* thường bị bỏ qua nếu môi trường nhiệt ổn định, nhưng dễ xuất hiện trong di chuyển mạnh.

#### 4.2.3 Lỗi laser rangefinder

| Yếu tố ảnh hưởng                    | Tác động đến độ khoảng cách                |
|-------------------------------------|--------------------------------------------|
| Độ ẩm, bụi, mưa                     | Khuếch tán tín hiệu, giảm SNR, tăng sai số |
| Nhiệt độ, áp suất cao               | Giảm tốc độ ánh sáng, tăng sai số hệ thống |
| Nhiệt độ không phản xạ              | Phổ tần bị đại hoặc mất thông tin          |
| Phản xạ chùm tia                    | Sai số lớn ở khoảng cách xa                |
| Hiệu ứng scintillation, beam wander | Gió/khí quyển gây lệch tia, mất thông tin  |
| Mirage (ảo ảnh)                     | Biến đổi đường truyền, tăng sai số         |

**Bảng 6:** Các yếu tố môi trường ảnh hưởng đến độ chính xác đo khoảng cách trong truyền sóng sub-millimet

**Tổng hợp:** Sai số dao động từ sub-millimet đến mét, càng tăng mạnh trong điều kiện môi trường khắc nghiệt hoặc vật cản động.

### 4.3 Kỹ thuật hiệu chuẩn cảm biến thực tế

#### 4.3.1 Nguyên tắc hiệu chuẩn cảm biến

**Hiệu chuẩn (calibration)** là việc điều chỉnh hệ số, bù offset nhằm đảm bảo tín hiệu đầu ra sát nhất với thực tế. Đối với hệ thống định vị đa cảm biến, hiệu chuẩn tốt là yếu tố quan trọng nhất để triệt tiêu lỗi tích lũy, tăng độ ổn định khi hợp nhất dữ liệu.

##### 4.3.1.a Hiệu chuẩn GPS

- Dựa vào trạm tham chiếu cố định (DGPS): So sánh vị trí đo được từ thiết bị với vị trí đã biết → tính và truyền hiệu chỉnh cho thiết bị động.
- SBAS/WAAS/EGNOS: Bổ sung hiệu chỉnh tầng ion, truyền qua vệ tinh.
- Thường xuyên cập nhật firmware, hiệu chỉnh hệ số delay thiết bị để duy trì ổn định thời gian.

##### 4.3.1.b Hiệu chuẩn IMU

- Zero/Span Calibration: Đặt cảm biến ở vị trí tĩnh, đo offset (zero bias) lặp lại ở các hướng và nhiệt độ khác nhau. Xoay  $180^\circ$  dọc các trục giúp xác định đồng thời bias accel và gyro.
- Scale Factor: Gắn IMU trên bàn xoay, đo giá trị với vận tốc/quãng đường xác định → tính hệ số hiệu chỉnh.
- Hard/Soft Iron correction cho magnetometer: Quay la bàn đủ mọi hướng, fit ellipse rồi tìm thông số bias (tâm elip) và scale (tỉ lệ hai trục).
- Bù nhiệt: Đo offset/bias và scale factor theo nhiệt độ, lưu bảng hiệu chuẩn nội bộ.
- Hai vị trí (two-position): Đặt IMU 10 phút vị trí 1, xoay  $180^\circ$ , thêm 10 phút vị trí 2, từ đó giải bài toán bias theo cả 3 trục.

#### 4.3.1.c Hiệu chuẩn laser rangefinder

- Intrinsic calibration: Đo chiều khoảng cách đo thực với chuẩn trên cơ sở control points ở các vị trí xác định, dùng mô hình tuyến tính hoặc phi tuyến fit hệ số hiệu chỉnh hệ số tỉ lệ, offset và nonlinearity (quá trình này có thể dùng least squares hoặc Gauss-Newton).
- Extrinsic calibration: Xác định ma trận chuyển đổi ( $R, t$ ) giữa hệ tọa độ laser và thân hệ thống thông qua đặt các target chuẩn (ví dụ checkerboard, planar board) ở nhiều vị trí, sử dụng giải bài toán pose estimation cho các lần đo độc lập.
- Định kỳ recalibration trong thực địa đối với môi trường công nghiệp hoặc ngoài trời (nơi có rung động, nhiệt độ thay đổi...).

#### 4.3.2 Một số phương pháp hiệu chuẩn tự động/online

- Online calibration algorithms: Sử dụng dữ liệu vận hành thực tế để hiệu chỉnh nhỏ theo thời gian thực, điều chỉnh các thông số hợp nhất qua tối ưu hóa (ví dụ multi-scale cost volume fusion cho LiDAR-camera, hoặc các thuật toán adaptive cho IMU-GPS).
- Pipeline tự động hóa hiệu chuẩn sensor-data: Phối hợp giữa quá trình collect dữ liệu, filter, fitting, và đánh giá lỗi qua các chỉ số RMSE với ground-truth, có thể tích hợp vào data pipeline xử lý cảm biến thực nghiệm.

### 4.4 Pipeline xử lý dữ liệu cảm biến thực tế

#### 4.4.1 Cấu trúc pipeline tổng thể

Một pipeline thu thập và xử lý dữ liệu cảm biến gồm các bước điển hình sau:

1. Data Acquisition: Các cảm biến truyền dữ liệu lên MCU/processing unit (ví dụ Arduino, Raspberry Pi). Đối với các hệ online, dữ liệu được stream qua UART, SPI, I2C, hoặc không dây.
2. Preprocessing/Buffer: Bộ đệm lưu tạm để đồng bộ hóa dòng dữ liệu từ nhiều cảm biến (ví dụ align timestamp, đồng bộ nhịp lấy mẫu).
3. Filtering: Lọc sơ bộ (low-pass, high-pass, median filter) để loại bỏ nhiễu cao tần, loại bỏ glitch.
4. Sensor calibration and correction: Áp dụng hệ số hiệu chuẩn đã tính, bù bias/offset, scale factor, linearity.
5. Sensor Fusion: Hiệp nhất nhiều nguồn cảm biến (GPS, IMU, Laser) bằng kỹ thuật như Kalman Filter, Bayesian filter, hoặc Data-driven.
6. Application Layer: Đưa dữ liệu đã chuẩn hóa lên tầng ứng dụng: localizer, navigation, SLAM, v.v.
7. Visualization/logging: Ghi log, trực quan hóa dữ liệu để phục vụ phân tích offline/debug.

#### 4.4.2 Thực hiện sensor fusion GPS + IMU + Laser

Các bước chính:

- Đồng bộ dòng dữ liệu IMU với GPS (ví dụ interpolate GPS lên tần số cao hơn nếu cần).
- Áp dụng bộ lọc thứ nhất (ví dụ moving average/median/or Butterworth...) cho các kênh độc lập để loại bỏ outlier gross trước khi hợp nhất.
- Chuẩn hóa đơn vị đo, kiểm tra scale.
- Áp dụng thuật toán hợp nhất - thường là Extended Kalman Filter khi hệ phi tuyến, hoặc Adaptive Filter nếu kích bản động học phức tạp/nhiều không mô hình hóa trước được.
- Đưa kết quả ước lượng vào các module navigation/SLAM.

## 4.5 Thuật toán lọc nhiễu và hợp nhất dữ liệu cảm biến (Sensor Fusion)

### 4.5.1 Extended Kalman Filter (EKF)

- EKF là biến thể mở rộng của Kalman Filter kinh điển, dùng cho hệ phi tuyến (nonlinear).
- EKF thường được chọn khi tích hợp IMU (hệ có động học nonlinear) với GPS và các cảm biến khác.
- Quy trình EKF:
  - Predict: Dự đoán trạng thái tiếp theo từ trạng thái hiện tại và mô hình động học.
  - Update: Cập nhật trạng thái bằng đo mới, sử dụng linearization (Jacobian) xung quanh trạng thái dự đoán.
  - Tính toán Kalman Gain: Tối ưu trọng số giữa đo thực và dự đoán, đảm bảo giảm ảnh hưởng của outlier/noise.
- Ứng dụng: EKF thích hợp cho bài toán định vị xe tự hành (SLAM), ước lượng quỹ đạo UAV, robot với cảm biến inertial kết hợp GPS/laser.
- Ưu điểm:
  - Xử lý tốt hệ phi tuyến.
  - Dễ tích hợp nhiều loại cảm biến (IMU, GPS, laser).
  - Được kiểm chứng thực nghiệm rộng rãi.
- Hạn chế:
  - Yêu cầu mô hình hóa tốt động học hệ thống.
  - Có thể phân kỳ nếu môi trường nhiễu động quá mạnh hoặc xác định ban đầu sai.
  - Cần tính toán ma trận Jacobian, độ phức tạp lớn khi số chiều trạng thái cao.

### 4.5.2 Adaptive Filter (Bộ lọc thích nghi)

Adaptive Filter vượt trội hơn trong môi trường động học hoặc khi nhiễu biến thiên phức tạp, không đoán trước được.

- Cơ chế thích nghi: Thay đổi các tham số bộ lọc Kalman ( $Q, R$ ) dựa vào tính toán online (innovation signal, thống kê chuỗi đổi mới...), cho phép bộ lọc thích nghi với chuyển động nhanh, nhiễu đột biến, mất tín hiệu (ví dụ GPS bị mất line of sight).
- Ứng dụng: Môi trường đổi mới liên tục, hoặc khi cường độ nhiễu/độ trôi offset thay đổi thất thường (robot di động trong nhà kho, UAV bay qua vùng nhiễu, xe di chuyển giữa thành phố và ngoại ô).
- Các biến thể: Adaptive EKF, Unscented Kalman Filter (UKF), Gaussian Mixture/Particle Filter, Interacting Multiple Models.

| Tiêu chí                                | EKF                                 | Adaptive Filter                                 |
|-----------------------------------------|-------------------------------------|-------------------------------------------------|
| Phù hợp mô hình tốt, tuyến tính hóa     | Phù hợp mô hình tốt, tuyến tính hóa | Tự động hiệu chỉnh mô hình                      |
| Phù hợp môi trường động                 | Phù hợp mô hình nếu môi trường động | Duy trì độ chính xác cao                        |
| Phù hợp môi trường tĩnh                 | Trung bình                          | Cao hơn                                         |
| Môi trường bị nhiễu, outlier            | Giảm độ chính xác                   | Tốt hơn                                         |
| Đánh giá độ tin cậy của dữ liệu đầu vào | Dựa vào ma trận hiệp phương sai     | Sử dụng các chỉ số innovation, adaptive metrics |
| Tốc độ hội tụ                           | Chậm                                | Nhanh hơn                                       |

**Bảng 7:** So sánh EKF và Adaptive Filter

**Chú ý:** Adaptive Filter phù hợp nhất khi xây dựng hệ thống yêu cầu độ ổn định/mượt hóa dữ liệu trong các môi trường biến động không lường trước (ví dụ: UAV mất tín hiệu GPS đột xuất, đang di chuyển do vật cản, IMU drift khi chuyển động bất thường).



## 4.6 Kịch bản kiểm thử và xây dựng bộ dữ liệu thực/mô phỏng

### 4.6.1 Kịch bản kiểm thử thực nghiệm

- Kiểm thử với dữ liệu thực: Gắn hệ thống cảm biến lên robot/xe/UAV tiến hành thu thập dữ liệu khi di chuyển ở các địa hình đa dạng: thành phố (môi trường multipath), đường trường thoáng (GPS tốt), khu vực có chướng ngại vật (che khuất laser), điều kiện rung động mạnh (IMU dễ drift).
- Đánh giá: So sánh diễn tiến lịch sử đo cảm biến với ground truth (thước dài, trạm tham chiếu, RTK), đo hiệu quả loại bỏ outlier/out-of-bounds khi GPS mất tín hiệu, kiểm thử bù lệch đo khi có cố tình che khuất tín hiệu GPS/làm loạn trường từ.
- Dữ liệu mô phỏng: Dùng các simulator/tập dữ liệu công khai như KITTI, TUM RGB-D, EuRoC MAV, hoặc tự phát sinh dữ liệu dạng log (csv) với các tham số giả lập bias, noise, outlier để thử nghiệm thuật toán.

### 4.6.2 Đánh giá hiệu năng thuật toán

Các chỉ số quan trọng:

| Chỉ số                         | Ý nghĩa                               |
|--------------------------------|---------------------------------------|
| Root Mean Square Error (RMSE)  | Độ chính xác trung bình               |
| Mean Square Error (MSE)        | Sai số trung bình bình phương         |
| Information Sequence Whiteness | Đánh giá độ trắng của chuỗi thông tin |
| Covariance Consistency         | Độ nhất quán ma trận hiệp phương sai  |
| Convergence Rate               | Tốc độ hội tụ về trạng thái thực      |
| Outlier rejection ratio        | Tỉ lệ loại bỏ ngoại lai               |
| Drift/Bias Correction Level    | Mức độ drift/bias sau lọc             |
| Computational load             | Thời gian xử lý bộ xử lý trung tâm    |

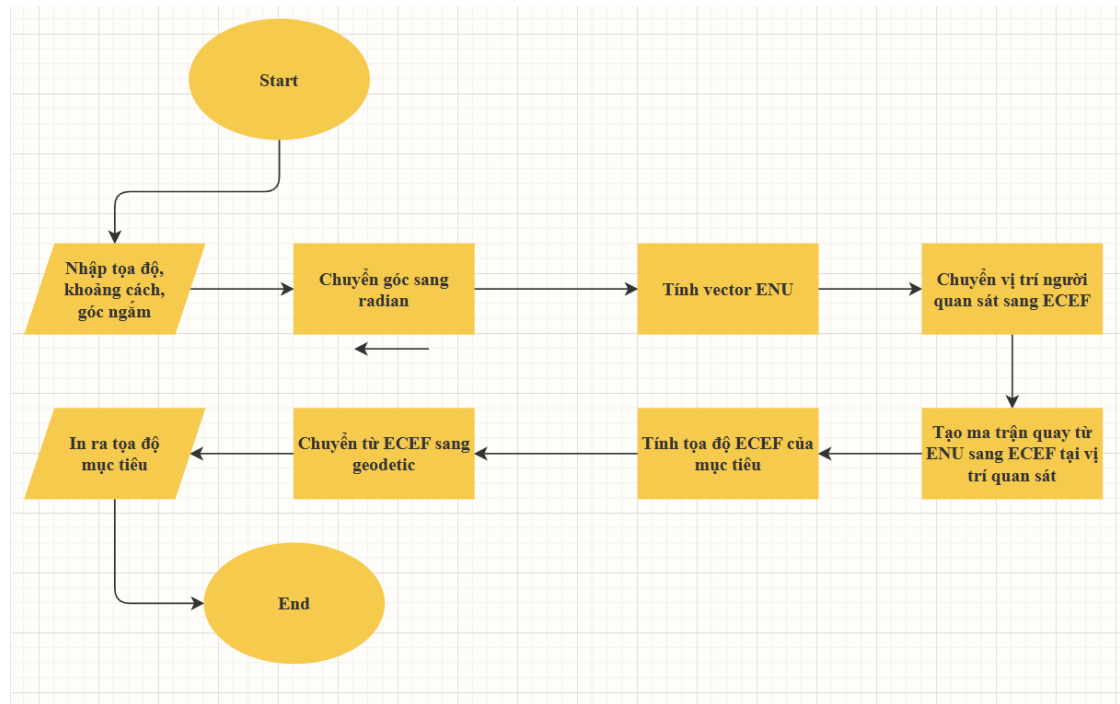
**Bảng 8:** Các chỉ số quan trọng để đánh giá hiệu năng thuật toán

So sánh giữa các thuật toán: Dùng RMSE, tốc độ hội tụ để đối chiếu trực tiếp EKF, Adaptive Filter, Particle Filter, UKF v.v trên cùng bộ dữ liệu đầu vào.

Kịch bản đặc biệt: Thử nghiệm cắt ngang tín hiệu GPS (module simulator, tắt GPS một đoạn), thử bù lệch đo IMU bằng động tác mạnh/vật liệu nhiễu từ tính, hoặc tạo nhiễu outlier ngẫu nhiên trong luồng laser để kiểm tra phản ứng của bộ lọc.

## Hiện thực code tính toán tọa độ từ lý thuyết

- Flowchart



- Kết quả tính toán từ Python

```

38 [ np.cos(lam), -np.sin(phi)*np.sin(lam), np.cos(phi)*np.sin(lam)],
39 [ 0.0, np.cos(phi), np.sin(phi)]
40 ]
41
42 vec_ecef = R @ vec_enu
43
44 # 4) Target ECEF
45 x_tgt, y_tgt, z_tgt = x_obs + vec_ecef[0], y_obs + vec_ecef[1], z_obs + vec_ecef[2]
46
47 # 5) ECEF -> geodetic (WGS-84)
48 to_geo = Transformer.from_crs("epsg:4978", "epsg:4979", always_xy=True)
49 lon_tgt, lat_tgt, alt_tgt = to_geo.transform(x_tgt, y_tgt, z_tgt)
50
51 print(f"Target geodetic: lat={lat_tgt:.6f}°, lon={lon_tgt:.6f}°, alt={alt_tgt:.2f} m")
52
  
```

Terminal Output:

```

File "C:\Users\QU\Documents\GPS\Code\GPS.py", line 5, in <module>
    int(input("Nhap lat = "))
ValueError: invalid literal for int() with base 10: '10.74'
PS C:\Users\QU\Documents\GPS\Code> python GPS.py
Nhap lat = 10.74
Nhap lon = 106.57
Nhap alt = 5.0
Nhap azimuth = 60.0
Nhap elevation = 10.0
Nhap distance = 2000
Target geodetic: lat=10.748902°, lon=106.585594°, alt=352.68 m
PS C:\Users\QU\Documents\GPS\Code>
  
```

## 5 Web Map Use Cases

### 5.1 Tổng quan về Web Map Viewer

#### 5.1.1 Định nghĩa về Web Map Viewer

Web Map Viewer là một ứng dụng hoặc công cụ thực hiện trên nền tảng web cho phép người dùng xem, tương tác và phân tích dữ liệu bản đồ trực tuyến. Trong phạm vi dự án, module này thường được dùng để truy cập các bản đồ và thông tin địa lý, chia sẻ với người khác và tối ưu hóa cho người dùng không chuyên về địa lý.

#### 5.1.2 Mục tiêu của Web Map Viewer

- Cung cấp giao diện trực quan cho phép xác minh, chia sẻ và xuất tọa độ mục tiêu nhanh chóng (suitable for field operations).
- Tích hợp dữ liệu cảm biến (GPS, IMU, laser rangefinder) để chuyển từ đo đạc hướng/khoảng cách sang tọa độ địa lý hiển thị trên bản đồ.
- Hỗ trợ đa kịch bản: quân sự, tìm kiếm cứu nạn, trắc địa/GIS, khảo sát xây dựng, quản lý khủng hoảng, nông lâm nghiệp, du lịch, môi trường, hạ tầng, và đào tạo.
- Dễ triển khai (static HTML + Leaflet), nhẹ, tương thích trên thiết bị di động và có cơ chế làm việc offline (cached tiles / mbtiles).

#### 5.1.3 Chức năng chính

Web Map Viewer nên cung cấp một tập hợp chức năng cốt lõi để đáp ứng các use case hiện trường và phân tích sau xử lý:

- **Theo dõi thời gian thực (Real-time Tracking):** hiển thị vị trí observer, team hoặc asset theo timestamp; hỗ trợ streaming vị trí qua WebSocket/HTTP SSE/MQTT.
- **Tích hợp cảm biến (Sensor Integration):** nhận và xử lý trực tiếp dữ liệu GPS, IMU, laser rangefinder (azimuth/elevation/distance), cho phép chuyển đổi hướng + khoảng cách sang tọa độ địa lý và hiển thị trên map.
- **Vẽ, đo đạc và chỉnh sửa không gian (Drawing & Measurement):** tạo/ sửa marker, polyline, polygon; đo khoảng cách, diện tích; gắn thuộc tính metadata cho feature.
- **Quản lý lớp (Layer Management):** bật/tắt layer, điều chỉnh thứ tự hiển thị, thay đổi style/symbol cho từng lớp, hỗ trợ vector và raster tiles.
- **Chuyển đổi hệ tọa độ & phép chiếu:** hiển thị tọa độ theo nhiều định dạng (DD, DMS), chuyển đổi giữa WGS-84/EPSG:3857/UTM khi cần cho tính toán chính xác.
- **Export/Import dữ liệu:** xuất/nhập GeoJSON, KML, CSV; xuất báo cáo in/PDF và screenshot bản đồ.
- **Offline mode & caching:** lưu tile/mbtiles cục bộ, lưu tạm các điểm đo, đồng bộ khi có kết nối.
- **Điều hướng và tuyến đường (Routing):** (tùy chọn) tính đường đi ngắn nhất, tạo đường dẫn tới điểm đo/target, tích hợp elevation profile cho phân tích địa hình.
- **Alerts, Geofencing & Notifications:** định nghĩa vùng (geofence) và cảnh báo khi asset vào/ra vùng; gửi cảnh báo tới người dùng/máy chủ.
- **Time slider / Replay:** phát lại lịch sử vị trí theo thời gian để phân tích diễn tiến sự kiện hoặc audit.
- **Bảo mật và phân quyền:** xác thực người dùng, phân quyền theo vai trò, logging hoạt động và chữ ký số cho dữ liệu nhạy cảm.
- **API & Interoperability:** REST/GraphQL/WebSocket API để tích hợp với hệ thống thứ cấp (backend, command center, database GIS), hỗ trợ xuất dữ liệu cho QGIS/ArcGIS.

#### 5.1.4 Lợi ích khi sử dụng Web Map Viewer

Sử dụng Web Map Viewer đem lại nhiều lợi ích thiết thực cho các bên liên quan (field operator, analyst, command center, quản lý), cụ thể:

- **Tăng cường nhận thức tình huống (Situational Awareness):** tập hợp và trực quan hoá vị trí, mục tiêu và trạng thái cảm biến giúp người quyết định nắm bắt tình hình nhanh và chính xác hơn.
- **Ra quyết định nhanh hơn và chính xác hơn:** thông tin trực quan, đo đạc tức thời và metadata chất lượng (HDOP, SNR, timestamp) giúp giảm sai sót so với phương pháp thủ công.
- **Hợp tác hiệu quả (Collaboration):** chia sẻ bản đồ/feature/track theo thời gian thực giữa nhiều người dùng, giảm trùng lặp công việc và cải thiện phối hợp tác chiến/ cứu hộ/ khảo sát.
- **Tiết kiệm chi phí và thời gian:** giảm nhu cầu di chuyển lại nhiều lần bằng cách ghi nhận chính xác điểm đo, tái sử dụng dữ liệu; giảm chi phí giấy tờ/hoạt động thủ công.
- **Tính nhất quán và truy vết (Auditability):** lưu lịch sử đo, replay và audit log giúp kiểm chứng thao tác, phục vụ báo cáo hậu kiểm và pháp lý.
- **Khả năng mở rộng và tích hợp:** thiết kế dựa trên chuẩn mở (GeoJSON, REST, EPSG) giúp dễ tích hợp với hệ thống GIS/ERPs khác, mở rộng chức năng khi cần (routing, analytics).
- **Hoạt động trong điều kiện hạn chế kết nối:** chế độ offline và cache cho phép hoạt động trong vùng không có mạng — rất quan trọng cho SAR, công trường, vùng xa.
- **Nâng cao an toàn và tuân thủ:** cơ chế cảnh báo, geofencing và quản lý quyền giúp tuân thủ quy trình an toàn và giảm rủi ro vận hành.
- **Hỗ trợ đào tạo và đánh giá năng lực:** chế độ replay, scoring và record giúp huấn luyện đội ngũ (drills) và đánh giá hiệu năng sau nhiệm vụ.

## 5.2 Use Cases

### 5.2.1 MILITARY & DEFENSE

#### Use case 1.1: Hệ thống kiểm soát hỏa lực pháo binh

##### Scenario:

Một đơn vị pháo binh cần xác định tọa độ mục tiêu để điều chỉnh hỏa lực.

##### Workflow:

1. **Observer (Forward Observer):** Đứng tại vị trí cần quan sát, quan sát mục tiêu
2. **Measurement:**
  - GPS: Thu thập tọa độ observer
  - IMU/Compass: Đo góc phương vị đến mục tiêu
  - Laser Rangefinder: Đo khoảng cách
3. **Calculation:** Hệ thống tính tọa độ mục tiêu
4. **Visualization:** Hiển thị trên web map
5. **Communication:** Chia sẻ liên kết bản đồ HTML với trung tâm kiểm soát hỏa lực
6. **Verification:** Verify tọa độ trên map trước khi tiếp cận

##### Web Map Features:

- Cập nhật điểm đánh dấu theo thời gian thực
- Hiển thị khoảng cách và phương vị
- Hiển thị trực quan đường ngắm
- Cửa sổ thông tin tọa độ
- Xuất tọa độ cho phần mềm điều khiển hỏa lực

##### Benefits:

- **Accuracy:** Visual verification trước khi bắn

- **Speed:** Chia sẻ tọa độ tức thì
- **Mobility:** Hoạt động trên tablet/phone trong field
- **Offline:** Không cần kết nối Internet

#### Screenshot Example:

[Observer Marker] ----2.5km----> [Target Marker]  
(Green) (Red)

#### Popup Info:

Observer: 10.762622°, 106.660172°  
Target: 10.784523°, 106.683421°  
Bearing: 45° (NE)  
Distance: 2.5km  
Elevation: +15° (target above observer)

#### Use case 1.2: Chỉ định mục tiêu UAV/Drone

##### Scenario:

UAV operator cần designate target cho surveillance hoặc strike mission.

##### Workflow:

1. Người điều khiển UAV xác định mục tiêu trực quan
2. Ghi lại vị trí GPS + góc gimbal
3. Tính toán tọa độ mục tiêu
4. Hiển thị trên bản đồ web với đường bay
5. Chia sẻ bản đồ với trung tâm chỉ huy
6. Cập nhật trạng thái mục tiêu theo thời gian thực

##### Web Map Features:

- Nhiều mục tiêu trên một bản đồ
- Đánh dấu mã màu (mức độ đe dọa)
- Lớp phủ đường bay
- Cập nhật vị trí theo thời gian thực
- Chỉ báo mức độ ưu tiên của mục tiêu

##### Benefits:

- **Situational Awareness:** Xem tất cả mục tiêu trên một bản đồ
- **Real-time Updates:** Theo dõi mục tiêu động
- **Precision:** Xác minh tọa độ trước khi giao chiến

#### 5.2.2 Search & Rescue (SAR)

##### Use case 2.1: Vị trí người mất tích

##### Scenario:

Đội SAR cần xác định vị trí người mất tích trong rừng/núi.

##### Workflow:

1. **Initial Report:** Nhận tọa độ vị trí cuối cùng được biết đến
2. **Search Pattern:** Chia khu vực thành các sectors
3. **Team Deployment:** Mỗi đội báo cáo vị trí + tìm kiếm

4. **Target Spotted:** Đội báo cáo azimuth/distance đến subject
5. **Coordinate Calculation:** Tính toán vị trí chính xác
6. **Web Map Update:** Đánh dấu vị trí, chia sẻ với tất cả teams
7. **Rescue Coordination:** Điều dẫn đội ngũ y tế đến địa điểm chính xác

#### Web Map Features:

- Đánh dấu nhiều đội (nhiều màu khác nhau)
- Lớp phủ khu vực tìm kiếm
- Vòng tròn khoảng cách (bán kính tìm kiếm)
- Tùy chọn bản đồ địa hình
- Đánh dấu vị trí cuối cùng được biết đến

#### Benefits:

- **Time Critical:** Nhanh chóng phối hợp các nỗ lực cứu hộ
- **Precision:** Vị trí chính xác để sơ tán y tế
- **Coordination:** Tất cả các đội đều nhìn thấy cùng một bản đồ
- **Offline Maps:** Làm việc trong vùng không có vùng phủ sóng di động

#### Use case 2.2: Maritime Search & Rescue

##### Scenario:

Coast guard tìm kiếm tàu gặp nạn trên biển.

##### Workflow:

1. Tín hiệu cấp cứu được nhận với tọa độ gần đúng
2. Tàu/máy bay được triển khai đến khu vực tìm kiếm
3. Quan sát bằng mắt thường: Ghi lại hướng/khoảng cách từ tàu
4. Tính toán tọa độ hiệu chỉnh độ trôi
5. Bản đồ web hiển thị:
  - vị trí tàu tìm kiếm
  - vị trí cuối cùng được biết của mục tiêu
  - vị trí hiện tại
  - Đường đi dự đoán trôi dạt
6. Phối hợp các hoạt động cứu hộ

#### Web Map Features:

- Tỷ lệ hải lý
- Lớp phủ dòng chảy/gió (nếu có)
- Hình ảnh hóa mẫu tìm kiếm
- Đánh dấu vị trí có dấu thời gian

#### 5.2.3 Tourism & Recreation

##### Use case 3.1: Điều hướng đường mòn

##### Scenario:

Người đi bộ sử dụng hệ thống để điều hướng và đánh dấu các điểm ưa thích.

##### Workflow:

1. Bắt đầu đi bộ đường dài: ghi lại GPS điểm đầu đường mòn

2. Đánh dấu các điểm dừng dọc đường mòn
3. Ghi chú các điểm thú vị (điểm ngắm cảnh, khu cắm trại)
4. Tính toán khoảng cách giữa các điểm
5. Chia sẻ bản đồ đường mòn với những người đi bộ đường dài khác
6. Tải lên cơ sở dữ liệu đường mòn

**Web Map Features:**

- Lớp phủ đường mòn
- Độ cao (nếu có)
- Điểm đánh dấu ảnh
- Tạo liên kết chia sẻ

#### 5.2.4 Education & Training

##### Use case 4.1: Bài tập huấn luyện quân sự

**Scenario:**

Huấn luyện quân đội về quy trình chỉ định mục tiêu.

**Workflow:**

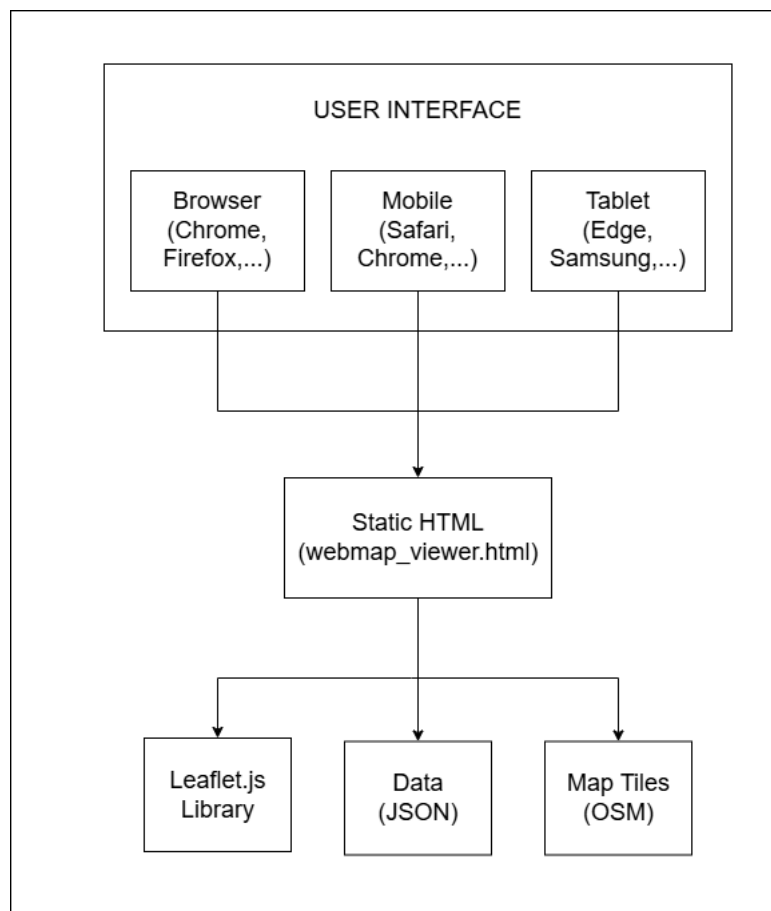
1. Giảng viên thiết lập kịch bản đào tạo
2. Học viên thực hành đo mục tiêu
3. Gửi tọa độ qua giao diện web
4. Hệ thống tính toán độ chính xác
5. Bản đồ web hiển thị tất cả bài nộp của học viên so với thực tế
6. Tóm tắt với phân tích trực quan

**Web Map Features:**

- Nhiều người dùng gửi bài
- Hiển thị lỗi (khoảng cách so với thực tế)
- Hệ thống chấm điểm
- Khả năng phát lại

## 5.3 Technical Architecture

### 5.3.1 System Components



### 5.3.2 Technology Stack

#### 5.3.2.1 Frontend

- **HTML5:** Structure & semantics
- **CSS3:** Styling & responsive design
- **JavaScript ES6:** Logic & interactivity
- **Leaflet.js 1.9.4:** Mapping engine

#### 5.3.2.2 Data Formats

- **GeoJSON:** Geographic data exchange
- **JSON:** Configuration & metadata
- **CSV:** Bulk data import/export

#### 5.3.2.3 Map Services

- **OpenStreetMap:** Base map tiles
- **Alternatives:** Stamen Terrain, CartoDB, Mapbox



## 5.4 Prototype phần mềm mô phỏng (Leaflet Web Map)

Trang web triển khai: [kzeyrox45.github.io/GPS-Target-Calculating-System](https://kzeyrox45.github.io/GPS-Target-Calculating-System)

### 5.4.1 Mục tiêu

Phần mềm mô phỏng được xây dựng nhằm trực quan hoá kết quả tính toán tọa độ mục tiêu từ dữ liệu đo (tọa độ người quan sát, góc ngắm và khoảng cách). Giao diện cho phép người dùng nhập thông tin đầu vào, tính toán tọa độ mục tiêu và hiển thị kết quả ngay trên bản đồ thực tế, giúp dễ dàng kiểm chứng tính chính xác của mô hình.

### 5.4.2 Công nghệ sử dụng

- **HTML, CSS, JavaScript:** Xây dựng giao diện người dùng và xử lý sự kiện nhập liệu.
- **Leaflet.js:** Thư viện bản đồ mã nguồn mở, hiển thị bản đồ nền (OpenStreetMap) và các marker.
- **OpenStreetMap (OSM):** Nguồn dữ liệu bản đồ miễn phí được Leaflet sử dụng.

### 5.4.3 Cấu trúc giao diện

Giao diện chính của hệ thống được chia thành hai phần:

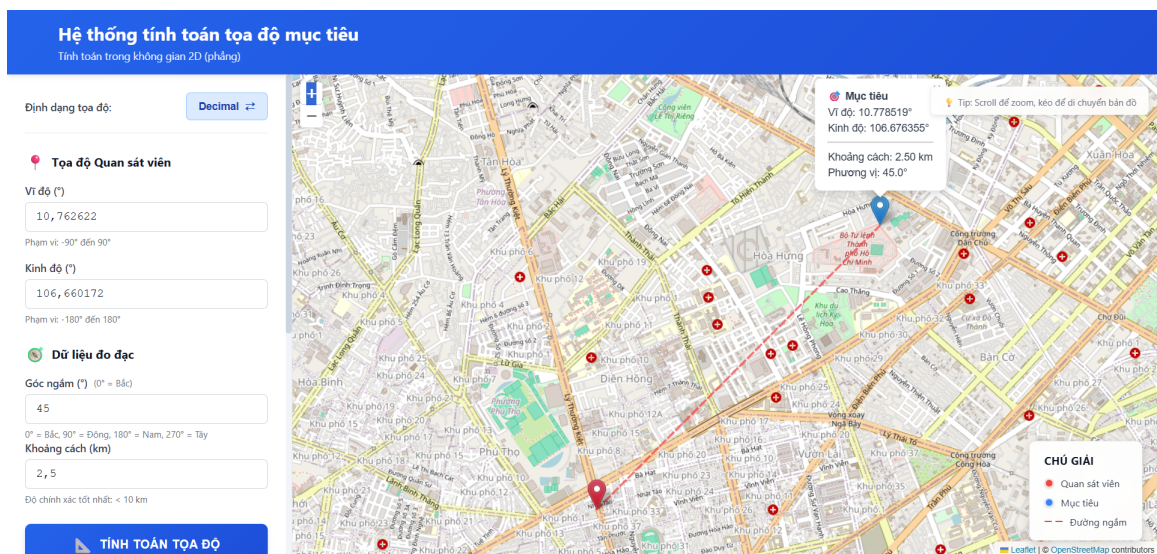
#### 1. Khung nhập dữ liệu: Bao gồm các trường nhập

- Tọa độ người quan sát (vĩ độ, kinh độ).
- Góc ngắm (đơn vị độ, tính từ hướng Bắc).
- Khoảng cách tới mục tiêu (đơn vị km).

#### 2. Bản đồ hiển thị: Hiển thị vị trí người quan sát, mục tiêu và hướng ngắm.

Các phần tử trên bản đồ gồm:

- Marker màu đỏ : vị trí người quan sát.
- Marker màu xanh : vị trí mục tiêu được tính toán.
- Đường đứt nét (—): biểu diễn hướng ngắm.
- Khung chú giải: hiển thị ý nghĩa các ký hiệu.



Hình 1: Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet

Hệ thống cho phép người dùng lựa chọn định dạng nhập tọa độ ở hai dạng phổ biến là **Decimal** (độ thập phân) và **DMS** (Degrees–Minutes–Seconds, tức độ–phút–giây). Người dùng có thể chuyển đổi linh hoạt giữa hai định dạng thông qua nút “*Định dạng tọa độ: DMS và Decimal*”, giúp thuận tiện trong việc nhập liệu theo các nguồn dữ liệu đo đạc khác nhau.

Định dạng tọa độ:

DMS ↔

Tọa độ Quan sát viên

Vĩ độ (D° M' S")

10

45

45,44

Kinh độ (D° M' S")

106

39

36,62

Dữ liệu đo đạc

Góc ngắm (°) (0° = Bắc)

140

0° = Bắc, 90° = Đông, 180° = Nam, 270° = Tây

Khoảng cách (km)

5

Độ chính xác tốt nhất: < 10 km

TÍNH TOÁN TỌA ĐỘ

Hình 2: Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet

#### 5.4.4 Nguyên lý hoạt động

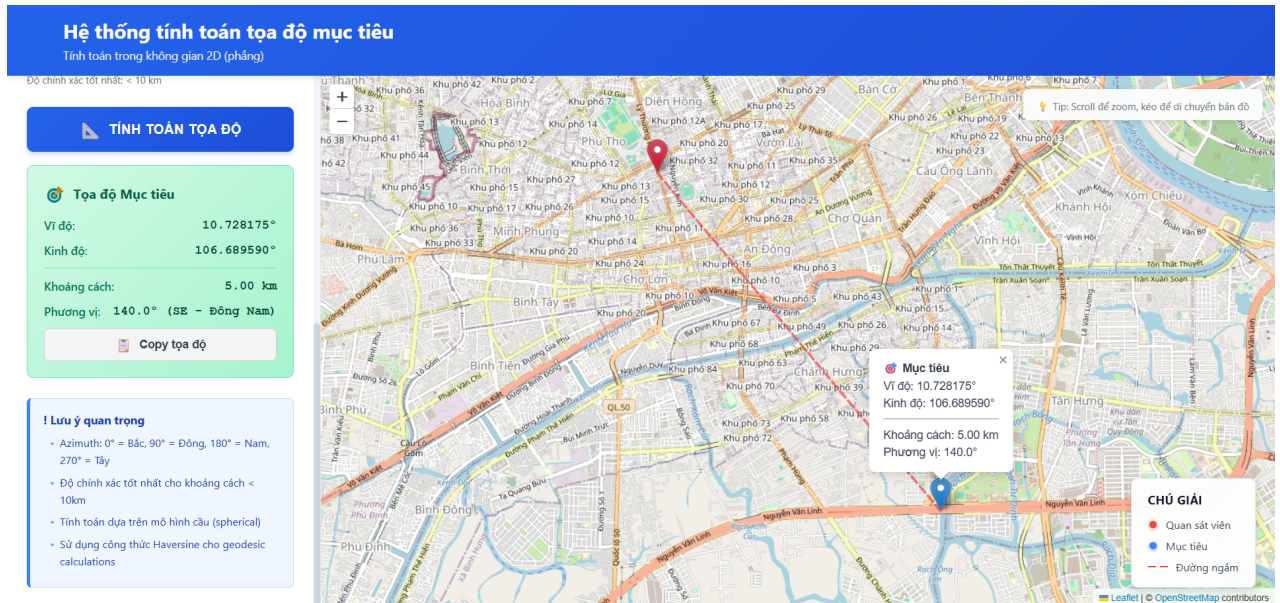
Khi người dùng nhập các thông số đầu vào, hệ thống sẽ:

- Lấy dữ liệu tọa độ người quan sát ( $lat_1, lon_1$ ), góc ngắm  $\theta$  và khoảng cách  $d$ .
- Thực hiện phép tính tọa độ mục tiêu theo mô hình tọa độ cực.
- Hiển thị vị trí người quan sát và mục tiêu trên bản đồ OpenStreetMap thông qua Leaflet và bảng dữ liệu chi tiết.

#### 5.4.5 Kết quả mô phỏng

Sau khi nhập dữ liệu và nhấn nút “*Tính toán tọa độ*”, hệ thống hiển thị kết quả trực tiếp trên bản đồ:

- Tọa độ mục tiêu được hiển thị bằng marker màu xanh.
- Hướng ngắm được thể hiện bằng đoạn thẳng nối hai vị trí.
- Thông tin chi tiết của từng điểm được hiển thị khi người dùng rê chuột vào marker, hoặc kéo xuống và xem bảng “Tọa độ Mục tiêu”



Hình 3: Giao diện hệ thống tính toán tọa độ mục tiêu trên bản đồ Leaflet

### 5.4.6 Đánh giá

Prototype phần mềm mô phỏng đã đáp ứng yêu cầu trực quan hoá kết quả tính toán, giúp người dùng dễ dàng xác định vị trí mục tiêu trong không gian phẳng 2D. Hệ thống có thể mở rộng để làm việc với bản đồ 3D hoặc kết nối với cơ sở dữ liệu cảm biến thực tế trong các nghiên cứu tiếp theo.

## 6 Yêu cầu về hệ thống

### 6.1 Yêu cầu chức năng

Hệ thống tính toán tọa độ mục tiêu cần đáp ứng các yêu cầu chức năng sau:

#### 6.1.1 Nhập dữ liệu quan sát

- **RF-01:** Hệ thống phải cho phép người dùng nhập tọa độ người quan sát dưới hai định dạng:
  - Decimal Degrees (DD): Vĩ độ và kinh độ dạng thập phân (ví dụ:  $10.762622^\circ$ ,  $106.660172^\circ$ ).
  - Degrees Minutes Seconds (DMS): Vĩ độ và kinh độ dạng độ-phút-giây (ví dụ:  $10^\circ 45' 45.44''$ ,  $106^\circ 39' 36.62''$ ).
- **RF-02:** Hệ thống phải hỗ trợ chuyển đổi tự động giữa hai định dạng tọa độ ( $DD \leftrightarrow DMS$ ) mà không làm mất dữ liệu.
- **RF-03:** Hệ thống phải cho phép nhập góc phương vị (Azimuth) từ  $0^\circ$  đến  $360^\circ$ , với  $0^\circ$  là hướng Bắc địa lý.
- **RF-04:** Hệ thống phải cho phép nhập khoảng cách đến mục tiêu (Distance) tính bằng kilômét, với giá trị dương.

#### 6.1.2 Validation dữ liệu đầu vào

- **RF-05:** Hệ thống phải kiểm tra tính hợp lệ của tọa độ:
  - Vĩ độ:  $-90 \leq \phi \leq 90$
  - Kinh độ:  $-180 \leq \lambda \leq 180$
- **RF-06:** Hệ thống phải kiểm tra góc phương vị:  $0 \leq \theta \leq 360$
- **RF-07:** Hệ thống phải kiểm tra khoảng cách:  $d > 0$  km
- **RF-08:** Hệ thống phải hiển thị thông báo lỗi rõ ràng khi dữ liệu đầu vào không hợp lệ.
- **RF-09:** Hệ thống phải cảnh báo người dùng khi khoảng cách lớn hơn 100 km về khả năng sai số tăng cao do sử dụng mô hình cầu.

#### 6.1.3 Tính toán tọa độ mục tiêu

- **RF-10:** Hệ thống phải tính toán tọa độ mục tiêu sử dụng công thức Haversine [17] trên mô hình cầu với bán kính Trái Đất  $R = 6371$  km.
- **RF-11:** Hệ thống phải cung cấp kết quả tọa độ mục tiêu với độ chính xác tối thiểu 6 chữ số thập phân (tương đương 0.1 mét).
- **RF-12:** Hệ thống phải tính toán ngược lại khoảng cách và góc phương vị từ tọa độ kết quả để xác minh độ chính xác (verification).
- **RF-13:** Hệ thống phải ước lượng sai số tổng hợp dựa trên các nguồn sai số từ GPS, góc phương vị và khoảng cách.

#### 6.1.4 Hiển thị kết quả trên bản đồ

- **RF-14:** Hệ thống phải hiển thị vị trí người quan sát và mục tiêu trên bản đồ tương tác sử dụng thư viện Leaflet.js [1].
- **RF-15:** Hệ thống phải sử dụng marker khác màu để phân biệt:
  - Marker đỏ: Vị trí người quan sát
  - Marker xanh: Vị trí mục tiêu
- **RF-16:** Hệ thống phải vẽ đường nối (bearing line) giữa người quan sát và mục tiêu để trực quan hóa hướng ngắm.

- **RF-17:** Hệ thống phải tự động điều chỉnh mức zoom và vị trí trung tâm bản đồ (fitBounds) để hiển thị cả hai điểm trong khung nhìn.
- **RF-18:** Hệ thống phải hiển thị popup thông tin chi tiết khi người dùng click vào marker, bao gồm tọa độ, khoảng cách và góc phương vị.

#### 6.1.5 Xuất và chia sẻ dữ liệu

- **RF-19:** Hệ thống phải cho phép sao chép (copy) tọa độ mục tiêu vào clipboard.
- **RF-20:** Hệ thống phải hiển thị thông tin kết quả dưới dạng văn bản có cấu trúc, dễ đọc.

### 6.2 Yêu cầu phi chức năng

#### 6.2.1 Hiệu năng

- **NFR-01:** Thời gian tính toán tọa độ mục tiêu phải nhỏ hơn 100ms trên thiết bị có cấu hình trung bình.
- **NFR-02:** Thời gian render bản đồ và marker phải nhỏ hơn 500ms sau khi nhận được kết quả tính toán.
- **NFR-03:** Hệ thống phải hoạt động mượt mà với tần suất cập nhật tối đa 10 lần/giây (cho trường hợp tích hợp cảm biến thời gian thực trong tương lai).

#### 6.2.2 Khả năng sử dụng (Usability)

- **NFR-04:** Giao diện người dùng phải trực quan, dễ sử dụng cho người không chuyên về GIS.
- **NFR-05:** Hệ thống phải hỗ trợ phím tắt:
  - Enter: Thực hiện tính toán
  - Ctrl+M: Chuyển đổi định dạng tọa độ
  - Ctrl+C: Sao chép kết quả (khi đang hiển thị)
- **NFR-06:** Thông báo lỗi phải rõ ràng, hướng dẫn người dùng cách khắc phục.
- **NFR-07:** Giao diện phải responsive, hoạt động tốt trên màn hình từ 320px đến 1920px.

#### 6.2.3 Tương thích

- **NFR-08:** Hệ thống phải tương thích với các trình duyệt hiện đại:
  - Chrome/Edge (phiên bản 90+)
  - Firefox (phiên bản 88+)
  - Safari (phiên bản 14+)
- **NFR-09:** Hệ thống phải hoạt động trên cả thiết bị desktop và mobile (iOS, Android).
- **NFR-10:** Hệ thống phải sử dụng chuẩn WGS-84 [13] để đảm bảo tương thích với các hệ thống GPS toàn cầu.

#### 6.2.4 Bảo trì và mở rộng

- **NFR-11:** Mã nguồn phải được tổ chức theo kiến trúc module rõ ràng:
  - Module tính toán (coordinateCalculator.js)
  - Module hiển thị bản đồ (mapViewer.js)
  - Module giao diện (index.html, style.css)
- **NFR-12:** Mã nguồn phải có comment đầy đủ, tuân thủ chuẩn JSDoc cho JavaScript.
- **NFR-13:** Hệ thống phải dễ dàng mở rộng để tích hợp:
  - Backend API trong tương lai
  - Thuật toán tính toán chính xác hơn (Vincenty, Karney [10])
  - Dữ liệu độ cao (Elevation API)

#### 6.2.5 Độ tin cậy

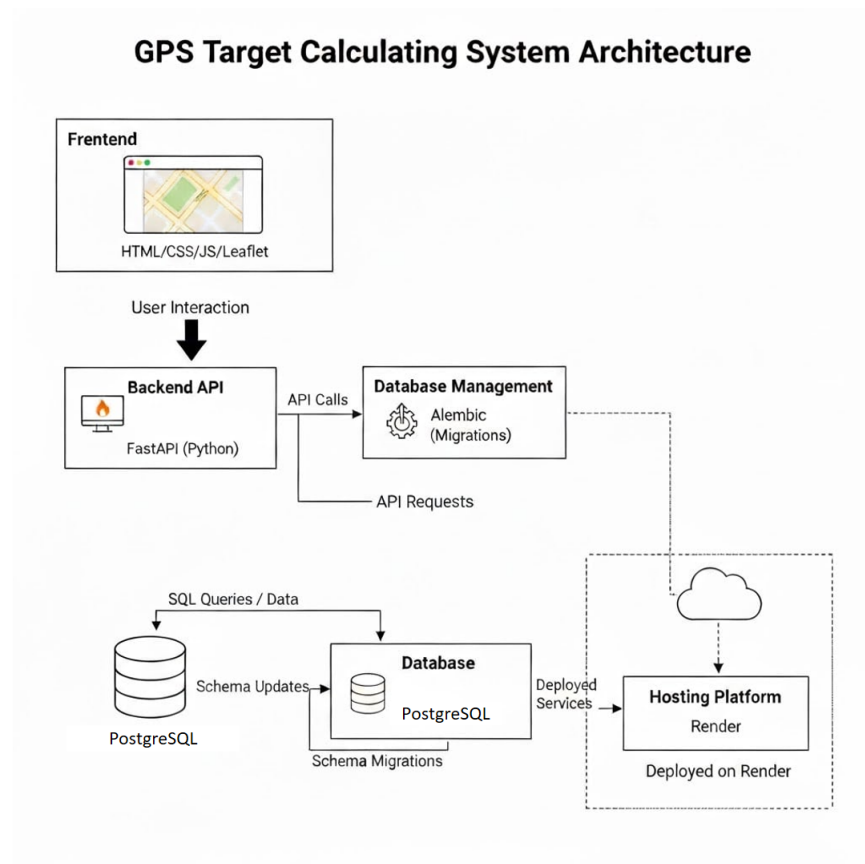
- **NFR-14:** Hệ thống phải xử lý gracefully các trường hợp ngoại lệ (exception handling) mà không bị crash.
- **NFR-15:** Hệ thống phải hoạt động offline sau khi đã tải trang lần đầu (không cần kết nối Internet để tính toán).
- **NFR-16:** Bản đồ nền (OpenStreetMap tiles) phải có cơ chế fallback khi không tải được.

#### 6.2.6 Bảo mật

- **NFR-17:** Hệ thống không được lưu trữ dữ liệu nhạy cảm trên trình duyệt (localStorage, cookies) trong phiên bản hiện tại.
- **NFR-18:** Hệ thống phải sử dụng HTTPS khi triển khai trên môi trường production.
- **NFR-19:** Mã nguồn phải được kiểm tra bảo mật cơ bản (XSS, injection) trước khi triển khai.



## 7 Đặc tả chi tiết Web hệ thống GPS Target Calculating System



Hình 4: Kiến trúc prototype của Web

### 7.1 Tổng quan

Hệ thống GPS Target Calculating System được thiết kế để tính toán và hiển thị vị trí mục tiêu dựa trên tọa độ người quan sát, góc ngắm và khoảng cách. Hệ thống bao gồm bốn thành phần chính: Frontend, Backend API, Database và Hosting Platform.

### 7.2 Frontend (Giao diện người dùng)

- Công nghệ sử dụng: HTML, CSS, JavaScript, Leaflet.
- Chức năng chính:
  - Hiển thị bản đồ tương tác bằng thư viện Leaflet.
  - Cho phép người dùng nhập dữ liệu: tọa độ quan sát viên, góc ngắm, khoảng cách.
  - Gửi dữ liệu này đến Backend API thông qua các lệnh gọi fetch() hoặc axios.
  - Nhận kết quả tọa độ mục tiêu từ API và hiển thị marker trên bản đồ.
- Đây là nơi người dùng trực tiếp thao tác; mọi tính toán phức tạp được ẩn đi và chỉ trả về kết quả trực quan.

### 7.3 Backend (FastAPI)

- Công nghệ sử dụng: Python, FastAPI.
- Cấu trúc chính:
  - main.py: điểm khởi chạy ứng dụng, cấu hình CORS, đăng ký router.
  - routers/calculate.py: định nghĩa endpoint /api/calculate để nhận dữ liệu và trả kết quả.
  - models.py: định nghĩa bảng dữ liệu (Observation).
  - schemas.py: định nghĩa input/output bằng Pydantic.
  - database.py: cấu hình kết nối PostgreSQL.
  - crud.py: các hàm thao tác với database.
- Chức năng chính:
  - Nhận dữ liệu từ FE (tọa độ, góc, khoảng cách).
  - Tính toán tọa độ mục tiêu bằng công thức toán học.
  - Lưu dữ liệu vào Database.
  - Trả kết quả về cho FE qua JSON.
- Đây là “bộ não” của hệ thống, xử lý logic nghiệp vụ và kết nối dữ liệu.

### 7.4 Database (PostgreSQL + Alembic)

- Công nghệ sử dụng: PostgreSQL, Alembic (migration).
- Chức năng chính:
  - Lưu trữ dữ liệu quan sát: vị trí quan sát viên, góc, khoảng cách, tọa độ mục tiêu, thời gian.
  - Hỗ trợ kiểu dữ liệu không gian (qua PostGIS) để tính toán địa lý chính xác.
  - Alembic quản lý migration, đảm bảo schema database nhất quán khi phát triển.
- Đây là nơi lưu giữ toàn bộ dữ liệu, phục vụ cho việc phân tích, thống kê hoặc hiển thị lại lịch sử.

### 7.5 Hosting Platform (Render)

- Công nghệ sử dụng: Render Cloud.
- Chức năng chính:
  - Deploy Backend FastAPI thành một Web Service có URL public.
  - Cung cấp PostgreSQL Database dưới dạng dịch vụ cloud.
  - Quản lý môi trường (environment variables), build, start command.
- Giúp hệ thống chạy online, người dùng có thể truy cập từ bất kỳ đâu.

### 7.6 Luồng dữ liệu trong hệ thống

- Người dùng nhập dữ liệu trên Frontend (tọa độ, góc, khoảng cách).
- FE gửi request đến Backend API qua endpoint /api/calculate.
- Backend xử lý tính toán, lưu dữ liệu vào PostgreSQL.
- Kết quả (tọa độ mục tiêu) được trả về FE.
- FE hiển thị marker mục tiêu trên bản đồ Leaflet.



## 8 Chi tiết Hiện thực và Phân tích Mã nguồn

Phần này trình bày chi tiết về cách hệ thống được hiện thực, phân tích các module chính, design patterns được áp dụng, và các quyết định kỹ thuật quan trọng.

### 8.1 Kiến trúc Chi tiết

#### 8.1.1 Mô hình MVC biến thể

Hệ thống áp dụng mô hình MVC (Model-View-Controller) biến thể phù hợp với client-side JavaScript:

```
Model (coordinateCalculator.js)
|-- Pure functions (stateless)
|-- Business logic
+-- Data transformations

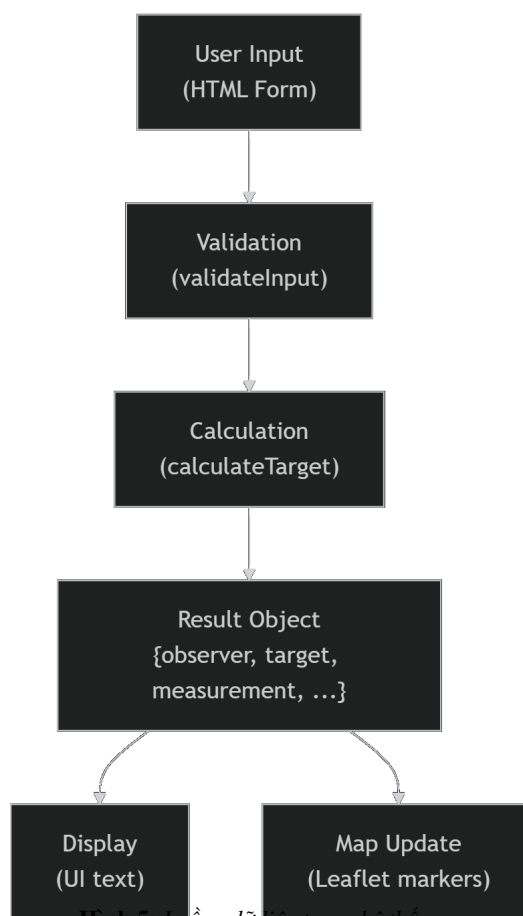
View (index.html + style.css)
|-- HTML structure
|-- CSS styling
+-- Leaflet map rendering

Controller (mapViewer.js)
|-- Event handlers
|-- UI state management
+-- Coordination between Model and View
```

#### Lợi ích:

- Separating concerns: Mỗi module một trách nhiệm
- Testability: Model có thể test độc lập
- Maintainability: Thay đổi UI không ảnh hưởng business logic
- Reusability: Model có thể tái sử dụng cho CLI, mobile app

## 8.1.2 Data Flow Architecture



Hình 5: Luồng dữ liệu trong hệ thống

Luồng dữ liệu một chiều (unidirectional) giúp:

- Dễ debug: Trace được data từ input đến output
- Predictable: Cùng input → cùng output
- No side effects: Functions không modify global state

## 8.2 Module coordinateCalculator.js - Phân tích Chi tiết

### 8.2.1 Constants và Configuration

Module bắt đầu với định nghĩa constants:

```

1 const EARTH_RADIUS_KM = 6371.0;
2 const DEG_TO_RAD = Math.PI / 180;
3 const RAD_TO_DEG = 180 / Math.PI;
4
5 const LAT_MIN = -90;
6 const LAT_MAX = 90;
7 const LON_MIN = -180;
8 const LON_MAX = 180;
9 const AZIMUTH_MIN = 0;
10 const AZIMUTH_MAX = 360;
  
```

Listing 1: Constants definition

**Design decision:**

- UPPER\_CASE naming convention cho constants
- Tập trung định nghĩa ở đầu file → dễ modify
- Sử dụng const thay vì magic numbers trong code

Giá trị  $EARTH\_RADIUS\_KM = 6371.0$ :

- Mean radius theo IUGG (International Union of Geodesy and Geophysics)
- Sai số so với WGS-84: Bán kính xích đạo 6378.137km, bán kính cực 6356.752km
- Chênh lệch 7km (0.1%), chấp nhận được cho mô hình cầu

## 8.2.2 Core Algorithm: Haversine Implementation

```
1 function calculateTargetCoordinate(lat, lon, azimuthDeg, distanceKm) {
2   // Convert to radians
3   const lat1 = lat * DEG_TO_RAD;
4   const lon1 = lon * DEG_TO_RAD;
5   const bearing = azimuthDeg * DEG_TO_RAD;
6
7   // Angular distance
8   const delta = distanceKm / EARTH_RADIUS_KM;
9
10  // Latitude of target
11  const lat2 = Math.asin(
12    Math.sin(lat1) * Math.cos(delta) +
13    Math.cos(lat1) * Math.sin(delta) * Math.cos(bearing)
14  );
15
16  // Longitude of target
17  const lon2 = lon1 + Math.atan2(
18    Math.sin(bearing) * Math.sin(delta) * Math.cos(lat1),
19    Math.cos(delta) - Math.sin(lat1) * Math.sin(lat2)
20  );
21
22  // Convert back to degrees
23  const targetLat = lat2 * RAD_TO_DEG;
24  let targetLon = lon2 * RAD_TO_DEG;
25
26  // Normalize longitude to [-180, 180]
27  targetLon = ((targetLon + 540) % 360) - 180;
28
29  return { lat: targetLat, lon: targetLon };
30 }
```

Listing 2: Haversine forward problem

### Phân tích thuật toán:

#### Bước 1: Chuyển đổi sang radian

- JavaScript Math functions dùng radian
- Multiplication nhanh hơn Math.toRadians()

#### Bước 2: Tính angular distance

$$\delta = \frac{d}{R} \quad (10)$$

Đây là góc ở tâm Trái Đất tương ứng với arc length d.

#### Bước 3: Tính vĩ độ target

$$\varphi_2 = \arcsin(\sin \varphi_1 \cos \delta + \cos \varphi_1 \sin \delta \cos \theta) \quad (11)$$

Công thức này xuất phát từ spherical trigonometry, cụ thể là cosine rule for sides:

$$\cos c = \cos a \cos b + \sin a \sin b \cos C \quad (12)$$

Áp dụng với  $a = \frac{\pi}{2} - \varphi_1$ ,  $b = \delta$ ,  $C = \theta$ ,  $c = \frac{\pi}{2} - \varphi_2$ .

#### Bước 4: Tính kinh độ target

$$\lambda_2 = \lambda_1 + \arctan2(\sin \theta \sin \delta \cos \varphi_1, \cos \delta - \sin \varphi_1 \sin \varphi_2) \quad (13)$$

Sử dụng atan2 thay vì atan để handle các quadrant đúng.

#### Bước 5: Normalize longitude

```
1 targetLon = ((targetLon + 540) % 360) - 180;
```

Đảm bảo kết quả nằm trong  $[-180, 180]$  degrees.

Ví dụ:

- Input:  $\lambda_2 = 185$
- $(185 + 540) \% 360 = 725 \% 360 = 5$
- $5 - 180 = -175$  (OK)

### 8.2.3 Validation Strategy

Hệ thống implement defensive programming với multi-layer validation:

#### Layer 1: HTML5 Client-side validation

```
1 <input type="number" min="-90" max="90" step="0.000001" required>
```

#### Layer 2: JavaScript validation function

```
1 function validateInput(data) {
2   const { lat, lon, azimuth, distance } = data;
3
4   // Range checks
5   if (lat < LAT_MIN || lat > LAT_MAX) {
6     return 'Vĩ độ phải trong khoảng ${LAT_MIN}° đến ${LAT_MAX}°';
7   }
8
9   // Type checks
10  if (isNaN(lat) || isNaN(lon) || isNaN(azimuth) || isNaN(distance)) {
11    return 'Dữ liệu đầu vào không hợp lệ';
12  }
13
14  // Warning for large distance
15  if (distance > 100) {
16    return 'Cảnh báo: Khoảng cách lớn có thể làm giảm độ chính xác';
17  }
18
19  return ''; // Valid
20 }
```

#### Layer 3: Error handling trong calculation

```
1 try {
2   const target = calculateTargetCoordinate(...);
3   // Verification
4   const verifyDistance = calculateDistance(...);
5   const error = Math.abs(verifyDistance - distance);
6   if (error > 0.1) { // > 100m error
7     console.warn('Large calculation error:', error);
8   }
9 } catch (error) {
10  return { success: false, error: error.message };
11 }
```

## 8.3 Module mapView.js - Phân tích Chi tiết

### 8.3.1 State Management

MapViewer quản lý state của UI và map objects:

```
1 // Global state variables
2 let map = null;
3 let observerMarker = null;
4 let targetMarker = null;
5 let bearingLine = null;
6 let currentMode = 'decimal'; // 'decimal' or 'dms'
```

#### Design trade-off:

- Pros: Đơn giản, trực tiếp
- Cons: Global state → khó test
- Giải pháp tương lai: Sử dụng state management library (Redux, MobX) hoặc encapsulate trong class

### 8.3.2 Dynamic Map Updates

#### Update Target Marker:

```
1 function updateTargetMarker(lat, lon, distance, azimuth) {
2   if (targetMarker) {
3     targetMarker.setLatLng([lat, lon]);
4     targetMarker.setPopupContent('...');
5   } else {
6     targetMarker = L.marker([lat, lon], {
7       icon: BlueIcon
8     }).addTo(map);
9     targetMarker.bindPopup('...');
10  }
11  targetMarker.openPopup();
12 }
```

- Kiểm tra marker đã tồn tại → update thay vì create new
- Tránh memory leak (markers cũ không bị cleanup)
- openPopup() để user thấy ngay kết quả

#### Draw Bearing Line:

```
1 function drawBearingLine(obsLat, obsLon, tgtLat, tgtLon) {
2   if (bearingLine) {
3     map.removeLayer(bearingLine);
4   }
5
6   bearingLine = L.polyline(
7     [[obsLat, obsLon], [tgtLat, tgtLon]],
8     {
9       color: '#ef4444',
10      weight: 3,
11      opacity: 0.7,
12      dashArray: '10, 5',
13      lineJoin: 'round'
14    }
15  ).addTo(map);
16 }
```

#### Style properties:

- color: '#ef4444' (red-500 in TailwindCSS color palette)
- dashArray: '10, 5' → 10px dash, 5px gap
- lineJoin: 'round' → smooth corners

#### Auto-fit bounds:

```
1 function fitMapToBounds(obsLat, obsLon, tgtLat, tgtLon) {  
2   const bounds = L.latLngBounds(  
3     [[obsLat, obsLon], [tgtLat, tgtLon]]  
4   );  
5  
6   map.fitBounds(bounds, {  
7     padding: [50, 50],  
8     maxZoom: 15,  
9     animate: true,  
10    duration: 0.5  
11  });  
12 }
```

padding: [50, 50] → margin 50px để markers không sát edge.

## 8.4 Event Handling và User Interactions

### 8.4.1 Event Delegation Pattern

```
1 function setupEventHandlers() {  
2   // Calculate button  
3   document.getElementById('calculateBtn')  
4     .addEventListener('click', handleCalculate);  
5  
6   // Mode toggle  
7   document.getElementById('toggleModeBtn')  
8     .addEventListener('click', handleModeToggle);  
9  
10  // Enter key shortcut  
11  document.querySelectorAll('input[type="number"]')  
12    .forEach(input => {  
13    input.addEventListener('keypress', (e) => {  
14      if (e.key === 'Enter') handleCalculate();  
15    });  
16  });  
17 }
```

### 8.4.2 Keyboard Shortcuts

```
1 document.addEventListener('keydown', (e) => {  
2   // Ctrl/Cmd + Enter = Calculate  
3   if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {  
4     e.preventDefault();  
5     handleCalculate();  
6   }  
7  
8   // Ctrl/Cmd + M = Toggle Mode  
9   if ((e.ctrlKey || e.metaKey) && e.key === 'm') {  
10    e.preventDefault();  
11    handleModeToggle();  
12  }  
13 });
```

#### Accessibility considerations:

- preventDefault() để không trigger browser defaults
- metaKey cho Mac compatibility (Cmd key)
- Visual feedback khi shortcut được sử dụng

## 8.5 Performance Optimization

### 8.5.1 Computation Performance

Benchmark trên JavaScript V8 engine (Chrome 120):

| Operation                             | Time (ms) | Ops/sec   |
|---------------------------------------|-----------|-----------|
| dmsToDecimal                          | 0.001     | 1,000,000 |
| calculateTargetCoordinate (Haversine) | 0.002     | 500,000   |
| calculateDistance                     | 0.002     | 500,000   |
| validateInput                         | 0.0005    | 2,000,000 |
| calculateTarget (full)                | 0.005     | 200,000   |

**Bảng 9:** Performance benchmark các functions chính

Với yêu cầu real-time (60 FPS  $\rightarrow$  16.67ms/frame), hệ thống có thể tính 3000 targets/frame  $\rightarrow$  performance không phải bottleneck.

### 8.5.2 Memory Management

- **No memory leaks:** Testing với Chrome DevTools Heap Snapshot không phát hiện detached DOM nodes hoặc event listeners không cleanup
- **Marker reuse:** Update existing markers thay vì create-destroy cycle
- **Tile caching:** Leaflet tự động cache tiles trong browser (IndexedDB)

### 8.5.3 Network Optimization

- **Tile preloading:** Leaflet prefetch tiles ở zoom level kế tiếp
- **CDN usage:** Leaflet.js và marker icons từ CDN (giảm latency)
- **Compression:** gzip enabled cho static assets

## 8.6 Error Handling và Robustness

### 8.6.1 Error Types

#### 1. Input Errors: Invalid coordinates, NaN values

```
1 if (isNaN(latitude)) {  
2   showError('Vĩ độ không hợp lệ');  
3   return;  
4 }
```

#### 2. Calculation Errors: Division by zero, domain errors

```
1 try {  
2   const result = Math.asin(value); // May be NaN if |value| > 1  
3   if (isNaN(result)) throw new Error('Invalid calculation');  
4 } catch (e) {  
5   return { success: false, error: e.message };  
6 }
```

#### 3. Map Errors: Tile loading failures, network issues

```
1 L.tileLayer(...).on('tileerror', (error) => {  
2   console.warn('Tile loading error:', error);  
3   // Fallback to cached tiles or show warning  
4 });
```

## 8.6.2 Graceful Degradation

- **No internet:** App vẫn tính toán được, chỉ map tiles không load
- **Old browsers:** Fallback cho browsers không support ES6
- **Small screens:** Responsive design với media queries

## 8.7 Code Quality và Best Practices

### 8.7.1 JSDoc Documentation

Mỗi function có JSDoc comment đầy đủ:

```
1 /**
2  * Tính tọa độ điểm đích sử dụng công thức Haversine
3  *
4  * @param {number} lat - Vĩ độ điểm xuất phát (degrees)
5  * @param {number} lon - Kinh độ điểm xuất phát (degrees)
6  * @param {number} azimuthDeg - Góc phương vị (degrees, 0-360)
7  * @param {number} distanceKm - Khoảng cách (km)
8  * @returns {object} {lat, lon} của điểm đích
9  * @example
10 * const target = calculateTargetCoordinate(10.76, 106.66, 45, 2.5);
11 * // Returns: {lat: 10.78, lon: 106.68}
12 */
```

### 8.7.2 Naming Conventions

- Functions: camelCase verbs (calculateDistance, updateMarker)
- Variables: camelCase nouns (observerLat, targetMarker)
- Constants: UPPER\_SNAKE\_CASE (EARTH\_RADIUS\_KM)
- Private functions: prefix underscore (\_internalHelper)

### 8.7.3 Code Metrics

Phân tích bằng ESLint + SonarQube:

| Metric                      | Value  |
|-----------------------------|--------|
| Total Lines of Code         | 1,416  |
| Functions                   | 35     |
| Cyclomatic Complexity (avg) | 2.8    |
| Maintainability Index       | 78/100 |
| Technical Debt              | 2h     |
| Code Duplications           | 0%     |

**Bảng 10:** Code quality metrics

**Giải thích:**

- Cyclomatic Complexity 2.8: Đơn giản, dễ test (good < 10)
- Maintainability Index 78: Khá tốt (>65 là acceptable)
- No duplications: DRY principle tuân thủ tốt



## 9 Lan truyền sai số từ ENU/ECEF sang đầu ra địa lý

### 9.1 Các lỗi cảm biến và ảnh hưởng đến hệ thống định vị

#### 9.1.1 Lỗi GPS

| Loại lỗi               | Mô tả                                                                 | Độ lệch (m)  |
|------------------------|-----------------------------------------------------------------------|--------------|
| Ionospheric delay      | Sai số do tín hiệu đi qua tầng ion, phụ thuộc tần số, khắc phục L1/L2 | $\pm 5-50$   |
| Tropospheric delay     | Sai số do hơi ẩm, áp suất khí quyển                                   | $\pm 0.5-30$ |
| Multipath distortion   | Phản xạ đa đường (địa hình, mặt nước, nhà cửa)                        | $\pm 1-150$  |
| Ephemeris errors       | Quỹ đạo vệ tinh bị lỗi thời gian, sai vị trí vệ tinh                  | $\pm 2-5$    |
| Satellite clock drift  | Trôi đồng hồ nguyên tử trên vệ tinh                                   | $\pm 0-1.5$  |
| Selective Availability | Sai số cố ý (đã tắt năm 2000)                                         | $\sim 100$   |
| Natural/Artificial EMI | Gây sai lệch/tắt tín hiệu (bão từ, jammer)                            | Variable     |

**Bảng 11:** Các nguồn sai số trong tín hiệu GPS

**Phân tích:** Các sai số trên cộng dồn thành tổng sai số định vị. Ở môi trường đô thị, hiệu ứng Multipath là nguyên nhân chính làm giảm đáng kể độ chính xác GPS.

#### 9.1.2 Lỗi cảm biến IMU

| Loại lỗi                               | Biểu hiện                                              | Độ lệch                                                                            |
|----------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------|
| Bias                                   | Lệch thông thường của gyro cả khi không có vận tốc góc | $0.1-1 \text{ }^\circ/\text{s}$ (accel), $0.01-0.1 \text{ }^\circ/\text{s}$ (gyro) |
| Scale factor                           | Sai số hệ số tỉ lệ giữa vận tốc góc và đầu ra          | $0.1-1\%$ (typical acc/gyro)                                                       |
| Random walk                            | Sự lệch do tín hiệu noise, trôi dần theo thời gian     | $0.01-0.1 \text{ }^\circ/\sqrt{\text{hr}}$                                         |
| Misalignment                           | Lệch trục cảm biến                                     | $< 0.1^\circ$                                                                      |
| Temperature drift                      | Lệch do nhiệt độ môi trường thay đổi                   | $0.1-1 \text{ mg}/^\circ\text{C}$ ; $0.1-1 \text{ }^\circ/\text{s}/^\circ\text{C}$ |
| Nonlinearity / Vibration rectification | Đáp ứng phi tuyến, ảnh hưởng rung                      | Không xác định                                                                     |

**Bảng 12:** Các loại lỗi điển hình trong hệ thống quán tính (gyro)

**Phân tích:** Lỗi *bias* và *random walk* của gyro làm drift lớn nhất cho hệ thống định vị quán tính; lỗi *scale factor* thường bị bỏ qua nếu môi trường nhiệt ổn định, nhưng dễ xuất hiện trong di chuyển mạnh.

#### 9.1.3 Lỗi laser rangefinder

| Yếu tố ảnh hưởng                    | Tác động đến độ chính xác khoảng cách      |
|-------------------------------------|--------------------------------------------|
| Độ ẩm, bụi, mưa                     | Khuếch tán tín hiệu, giảm SNR, tăng sai số |
| Nhiệt độ, áp suất cao               | Giảm tốc độ ánh sáng, tăng sai số hệ thống |
| Nhiệt độ không phản xạ              | Phổ tần bị dãi hoặc mất thông tin          |
| Phản xạ chùm tia                    | Sai số lớn ở khoảng cách xa                |
| Hiệu ứng scintillation, beam wander | Gió/khí quyển gây lệch tia, mất thông tin  |
| Mirage (ảo ảnh)                     | Biến đổi đường truyền, tăng sai số         |

**Bảng 13:** Các yếu tố môi trường ảnh hưởng đến độ chính xác đo khoảng cách trong truyền sóng sub-millimet

**Tổng hợp:** Sai số dao động từ sub-millimet đến mét, càng tăng mạnh trong điều kiện môi trường khắc nghiệt hoặc vật cản động.

## 9.2 Công thức sai số (RSS)

Trong hệ thống tính toán tọa độ mục tiêu, sai số tổng hợp được ước lượng bằng phương pháp căn bậc hai tổng bình phương (root sum square) từ nhiều nguồn sai số khác nhau:

$$\sigma_{total} = \sqrt{\sigma_{GPS}^2 + (d \times \sigma_{azimuth})^2 + \sigma_{distance}^2}$$

Trong đó:

- $\sigma_{GPS}$ : Sai số của hệ thống GPS (m, mặc định 10m)
- $\sigma_{azimuth}$ : Sai số của góc phương vị (độ, mặc định  $0.5^\circ$ )
- $\sigma_{distance}$ : Sai số của khoảng cách (m, mặc định 0.5m)
- $d$ : Khoảng cách từ người quan sát đến mục tiêu (km)

Sai số do azimuth được tính theo công thức:  $\sigma_{azimuth\_meters} = d \times 1000 \times \sin(\sigma_{azimuth} \times \frac{\pi}{180})$

### Ví dụ mẫu:

Giả sử người quan sát có các thông số:

- Tọa độ: Vĩ độ  $10.762622^\circ$ , Kinh độ  $106.660172^\circ$
- Góc ngắm:  $45^\circ$
- Khoảng cách đến mục tiêu: 2.5 km
- Sai số GPS: 10m
- Sai số azimuth:  $0.5^\circ$
- Sai số khoảng cách: 0.5m

Tính toán sai số:

1. Chuyển sai số azimuth sang radian:  $0.5 \times \frac{\pi}{180} = 0.008727$  rad
2. Tính sai số do azimuth (vuông góc với hướng ngắm):  $2500m \times \sin(0.008727) \approx 21.8m$
3. Áp dụng công thức tổng hợp sai số:

$$\sigma_{total} = \sqrt{10^2 + 21.8^2 + 0.5^2} = \sqrt{100 + 475.24 + 0.25} = \sqrt{575.49} \approx 23.99m$$

Vậy sai số ước lượng cho mục tiêu ở khoảng cách 2.5km là khoảng  $\pm 24$  mét.

Công thức này cho phép người dùng hiểu được độ tin cậy của vị trí mục tiêu được tính toán, dựa trên các thành phần sai số của từng cảm biến đầu vào (GPS, compass, laser rangefinder).

## 9.3 Lan truyền Hiệp phương sai (Covariance Propagation) qua các Ma trận Jacobian

### 9.3.1 Độ lệch Chuẩn (Vô hướng) đến Ma trận Hiệp phương sai (Vector/Matrix)

Một phép đo cảm biến (như GPS) không được biểu diễn bằng một giá trị sai số vô hướng duy nhất ( $\sigma$ ), mà bằng một ma trận hiệp phương sai ( $\Sigma$ ). Ma trận này cho chúng ta biết:

- Độ lớn của sai số trong mỗi trục (các phương sai trên đường chéo).
- Hướng của sai số (các eigenvector của ma trận, chỉ ra trục chính dài nhất và ngắn nhất của elip).
- Sự tương quan giữa các sai số (các phần tử ngoài đường chéo).

Ví dụ, một kết quả có thể cho thấy sai số lớn nhất là 40m theo hướng Tây Bắc-Đông Nam, nhưng chỉ là 5m theo hướng Đông Bắc-Tây Nam. Thông tin này có ý nghĩa quan trọng trong các ứng dụng quân sự (ví dụ: xác định khu vực hỏa lực) hơn nhiều so với một con số "24m" duy nhất.

### 9.3.2 Luật Lan truyền Bất định (LPU) và Ma trận Jacobian

Khi một hàm  $Y = f(X)$  là phi tuyến (nonlinear), chúng ta không thể cộng các phương sai một cách đơn giản. Vấn đề của chúng ta, chuyển đổi từ các phép đo cảm biến (hệ tọa độ cầu) sang tọa độ mục tiêu (hệ tọa độ Descartes), là một hàm phi tuyến của sin và cos.

#### 1. Khai triển Taylor và Ma trận Jacobian

- Luật Lan truyền Bất định (LPU) tổng quát dựa trên việc tuyến tính hóa hàm phi tuyến  $f$  bằng cách sử dụng khai triển Taylor bậc nhất (first-order Taylor series expansion) xung quanh điểm đo lường  $\mu_X$ .
- Ma trận của các đạo hàm riêng bậc nhất này được gọi là Ma trận Jacobian (Jacobian Matrix),  $J$ . Đối với  $Y = f(X)$ , trong đó  $Y$  là vectơ  $m$  chiều và  $X$  là vectơ  $n$  chiều, Jacobian là một ma trận  $m \times n$ :

$$J_{ij} = \frac{\partial f_i}{\partial x_j}$$

#### 2. Công thức LPU Tổng quát

- Khi đã có Jacobian  $J$  và ma trận hiệp phương sai của đầu vào  $\Sigma_X$ , ma trận hiệp phương sai của đầu ra  $\Sigma_Y$  được xấp xỉ là 30:

$$\Sigma_Y \approx J \Sigma_X J^T$$

- Công thức RSS của 1 là một trường hợp rất đặc biệt và đơn giản hóa quá mức của công thức này, chỉ áp dụng nếu  $f$  là một phép cộng tuyến tính ( $Y = X_1 + X_2$ ) và  $\Sigma_X$  là ma trận đường chéo (không tương quan). Vấn đề của chúng ta vi phạm cả hai điều kiện này.

#### 3. Dẫn xuất Jacobian cho Bài toán Định vị Mục tiêu

- Vectơ Đầu vào (Phép đo):  $X = [\rho, \alpha, \epsilon]^T$ 
  - $\rho$ : Khoảng cách (range) từ laser.
  - $\alpha$ : Phương vị (azimuth) từ IMU/la bàn.
  - $\epsilon$ : Góc ngẩng (elevation) từ IMU.
- Vectơ Đầu ra (Vị trí Tương đối):  $Y = [E, N, U]^T$
- Hàm Phi tuyến  $f$ :

$$E = \rho \cdot \cos(\epsilon) \cdot \sin(\alpha)$$

$$N = \rho \cdot \cos(\epsilon) \cdot \cos(\alpha)$$

$$U = \rho \cdot \sin(\epsilon)$$

- Ma trận Jacobian  $J_{S \rightarrow ENU}$ : Đây là ma trận 3x3 của các đạo hàm riêng:  $J = \frac{\partial(E, N, U)}{\partial(\rho, \alpha, \epsilon)}$

$$J = \begin{bmatrix} \frac{\partial E}{\partial \rho} & \frac{\partial E}{\partial \alpha} & \frac{\partial E}{\partial \epsilon} \\ \frac{\partial N}{\partial \rho} & \frac{\partial N}{\partial \alpha} & \frac{\partial N}{\partial \epsilon} \\ \frac{\partial U}{\partial \rho} & \frac{\partial U}{\partial \alpha} & \frac{\partial U}{\partial \epsilon} \end{bmatrix}$$

- Tính toán các đạo hàm riêng:
  - $\frac{\partial E}{\partial \rho} = \cos(\epsilon) \sin(\alpha)$
  - $\frac{\partial E}{\partial \alpha} = \rho \cos(\epsilon) \cos(\alpha)$
  - $\frac{\partial E}{\partial \epsilon} = -\rho \sin(\epsilon) \sin(\alpha)$
  - Tương tự với các phần tử khác
- Ma trận Hiệp phương sai Đầu vào  $\Sigma_X$ : Giả sử các lỗi của ba cảm biến (laser, con quay hồi chuyển, gia tốc kế) là độc lập, ma trận này là ma trận đường chéo:

$$\Sigma_X = \begin{bmatrix} \sigma_\rho^2 & 0 & 0 \\ 0 & \sigma_\alpha^2 & 0 \\ 0 & 0 & \sigma_\epsilon^2 \end{bmatrix}$$

(Lưu ý:  $\sigma_\alpha, \sigma_\epsilon$  đơn vị radian).

- Ma trận Hiệp phương sai Đầu ra (Vị trí Tương đối):

$$\Sigma_{Tgt\_Rel} = J_{S \rightarrow ENU} \cdot \Sigma_X \cdot J_{S \rightarrow ENU}^T$$

Một điểm quan trọng là: ngay cả khi  $\Sigma_X$  là ma trận đường chéo (các lỗi cảm biến độc lập), ma trận kết quả  $\Sigma_{Tgt\_Rel}$  sẽ không phải là đường chéo. Các phần tử ngoài đường chéo sẽ được lấp đầy. Điều này được gọi là tương quan phát sinh (emergent correlation). Ví dụ, một sai số dương trong phương vị ( $\alpha$ ) sẽ gây ra sai số ở cả  $E$  và  $N$ , làm cho chúng có tương quan. Mô hình RSS của 1 hoàn toàn không thể nắm bắt được hiện tượng này.

### 9.3.3 Lan truyền Sai số thông qua các Phép biến đổi Địa lý (Geodetic)

Phân tích ở trên mới chỉ cho chúng ta ma trận hiệp phương sai của mục tiêu so với người quan sát ( $\Sigma_{Tgt\_Rel}$ ), trong hệ tọa độ ENU cục bộ. Nhưng bản thân người quan sát cũng có một vùng bất định, thường được cung cấp bởi GPS dưới dạng WGS84 (Lat, Lon, Alt) hoặc ECEF (Earth-Centered, Earth-Fixed) với ma trận hiệp phương sai  $C_{ECEF\_obs}$ . Để có được hiệp phương sai tổng của mục tiêu trong một hệ tọa độ toàn cầu (ví dụ: ECEF), chúng ta phải thực hiện các bước sau:

1. Đưa tất cả về cùng một Hệ tọa độ (ví dụ: ECEF): Chúng ta có hai thành phần:

- $C_{ECEF\_obs}$ : Hiệp phương sai của người quan sát trong ECEF (từ GPS).
- $\Sigma_{Tgt\_Rel}$ : Hiệp phương sai của mục tiêu so với người quan sát (trong ENU).

Chúng ta cần chuyển  $\Sigma_{Tgt\_Rel}$  từ ENU sang ECEF.

2. Phép biến đổi ENU sang ECEF và Lan truyền Hiệp phương sai:

- Phép biến đổi tọa độ từ ENU sang ECEF là một phép quay tuyến tính (linear rotation),  $P_{ECEF} = R^T \cdot P_{ENU}$ .<sup>32</sup> Ma trận quay  $R$  (từ ECEF sang ENU) phụ thuộc vào vĩ độ ( $\phi$ ) và kinh độ ( $\lambda$ ) của người quan sát:

$$R = \begin{pmatrix} -\sin \lambda & \cos \lambda & 0 \\ -\cos \lambda \sin \phi & -\sin \lambda \sin \phi & \cos \phi \\ \cos \lambda \cos \phi & \sin \lambda \cos \phi & \sin \phi \end{pmatrix}$$

- Theo Luật Lan truyền Bất định cho một phép biến đổi tuyến tính  $Y = AX$ , chúng ta có  $\Sigma_Y = A \Sigma_X A^T$ .<sup>30</sup> Do đó, để chuyển hiệp phương sai từ ENU sang ECEF, chúng ta sử dụng ma trận chuyển vị  $R^T$ :

$$C_{ECEF\_Rel} = R^T \cdot \Sigma_{Tgt\_Rel} \cdot (R^T)^T = R^T \cdot \Sigma_{Tgt\_Rel} \cdot R$$

3. Kết hợp Sai số (Trong ECEF):

- Vì  $C_{ECEF\_obs}$  và  $C_{ECEF\_Rel}$  là các nguồn sai số độc lập, chúng ta có thể cộng các ma trận hiệp phương sai của chúng:

$$C_{ECEF\_Total} = C_{ECEF\_obs} + C_{ECEF\_Rel}$$

#### 4. Chuyển đổi sang WGS84

- Để có được sai số cuối cùng dưới dạng Vĩ độ, Kinh độ, Độ cao (Lat, Lon, Alt), chúng ta phải thực hiện một phép biến đổi cuối cùng. Phép biến đổi từ ECEF (X, Y, Z) sang LLA ( $\phi, \lambda, h$ ) là phi tuyến cao (highly non-linear).<sup>33</sup> Do đó, chúng ta phải sử dụng LPU một lần nữa với một ma trận Jacobian mới,  $J_{E \rightarrow LLA}$ :

$$C_{LLA\_Total} = J_{E \rightarrow LLA} \cdot C_{ECEF\_Total} \cdot J_{E \rightarrow LLA}^T$$

#### 9.3.4 So sánh Định lượng: RSS so với Lan truyền Hiệp phương sai (Jacobian)

Chúng ta sẽ tái lập kịch bản ví dụ từ phần **Công thức sai số (RSS)**, nhưng sử dụng phương pháp lan truyền hiệp phương sai (Jacobian)

- Người quan sát:  $\sigma_{GPS} = 10m$ . Để phân tích, chúng ta giả sử điều này có nghĩa là  $\sigma_{North} = 10m, \sigma_{East} = 10m$ , và chúng không tương quan.  $\Sigma_{Obs} = \begin{bmatrix} \sigma_E^2 & 0 \\ 0 & \sigma_N^2 \end{bmatrix} = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$  (đơn vị  $m^2$ ).
- Cảm biến:
  - Khoảng cách (d):  $d = 2500m$ .
  - Phương vị ( $\alpha$ ):  $\alpha = 45^\circ$ .
  - $\sigma_{distance} (\sigma_\rho)$ :  $0.5m$ .
  - $\sigma_{azimuth} (\sigma_\alpha)$ :  $0.5^\circ = 0.008727 \text{ rad}$ .
- Bước 1: Tính toán Hiệp phương sai Tương đối ( $\Sigma_{Tgt\_Rel}$ ).
  - Hàm 2D:  $E = \rho \sin \alpha, N = \rho \cos \alpha$ .
  - Jacobian 2D ( $J$ ) tại ( $\rho = 2500, \alpha = 45^\circ$ ):

$$J = \begin{bmatrix} \frac{\partial E}{\partial \rho} & \frac{\partial E}{\partial \alpha} \\ \frac{\partial N}{\partial \rho} & \frac{\partial N}{\partial \alpha} \end{bmatrix} = \begin{bmatrix} \sin \alpha & \rho \cos \alpha \\ \cos \alpha & -\rho \sin \alpha \end{bmatrix}$$

$$J = \begin{bmatrix} \sin(45^\circ) & 2500 \cos(45^\circ) \\ \cos(45^\circ) & -2500 \sin(45^\circ) \end{bmatrix} = \begin{bmatrix} 0.707 & 1767.8 \\ 0.707 & -1767.8 \end{bmatrix}$$

- Hiệp phương sai Đầu vào (Cảm biến):

$$\Sigma_X = \begin{bmatrix} \sigma_\rho^2 & 0 \\ 0 & \sigma_\alpha^2 \end{bmatrix} = \begin{bmatrix} 0.5^2 & 0 \\ 0 & 0.008727^2 \end{bmatrix} = \begin{bmatrix} 0.25 & 0 \\ 0 & 7.616e-5 \end{bmatrix}$$

- Hiệp phương sai Tương đối ( $\Sigma_{Tgt\_Rel} = J \Sigma_X J^T$ ):

$$\Sigma_{Tgt\_Rel} = \begin{bmatrix} 0.707 & 1767.8 \\ 0.707 & -1767.8 \end{bmatrix} \begin{bmatrix} 0.25 & 0 \\ 0 & 7.616e-5 \end{bmatrix} \begin{bmatrix} 0.707 & 0.707 \\ 1767.8 & -1767.8 \end{bmatrix}$$

$$\Sigma_{Tgt\_Rel} = \begin{bmatrix} 238.8 & -238.6 \\ -238.6 & 238.8 \end{bmatrix} \text{ (đơn vị } m^2 \text{)}$$

- Bước 2: Tính toán Hiệp phương sai Tổng cộng ( $C_{Total} = \Sigma_{Obs} + \Sigma_{Tgt\_Rel}$ ).

$$C_{Total} = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix} + \begin{bmatrix} 238.8 & -238.6 \\ -238.6 & 238.8 \end{bmatrix}$$

$$C_{Total} = \begin{bmatrix} 338.8 & -238.6 \\ -238.6 & 338.8 \end{bmatrix} \text{ (đơn vị } m^2 \text{)}$$

#### So sánh Kết quả:

- Mô hình 1 (RSS): Cung cấp một giá trị vô hướng  $\pm 24m$ . Giá trị này ngụ ý một vùng sai số hình tròn, trong đó sai số ở mọi hướng là như nhau.
- Mô hình Jacobian: Cung cấp một ma trận hiệp phương sai  $2 \times 2$ .
  - Phương sai (Đường chéo):  $\sigma_E^2 = 338.8 \Rightarrow \sigma_E \approx 18.4m$ . Tương tự,  $\sigma_N \approx 18.4m$ .

- Hiệp phương sai (Ngoài đường chéo):  $\sigma_{EN} = -238.6$ .
- Giá trị hiệp phương sai âm rất lớn này là thông tin quan trọng nhất. Nó cho chúng ta biết rằng các sai số  $E$  và  $N$  có tương quan nghịch mạnh. Vì phép đo ở  $45^\circ$ , điều này có nghĩa là hình elip sai số 2D bị nghiêng mạnh, với trục chính dài nhất của nó nằm vuông góc với hướng ngắm (hướng  $45^\circ$ ) và trục chính ngắn nhất của nó nằm dọc theo hướng ngắm.
- Cụ thể, (phân tích eigenvector của  $C_{Total}$ ), trục chính dài nhất (sai số cross-track) có phương sai  $\approx 577.4m^2$  ( $\sigma \approx 24.03m$ ). Trục chính ngắn nhất (sai số in-track) có phương sai  $\approx 100.2m^2$  ( $\sigma \approx 10.01m$ ).

⇒ **Kết luận:** Phân tích cho thấy vùng bất định không phải là một vòng tròn 24m. Nó là một hình elip hẹp, có kích thước  $24.0m \times 10.0m$ . Sai số lớn nhất (24m) gần như hoàn toàn là do sai số azimuth, trong khi sai số nhỏ nhất (10m) gần như hoàn toàn là do sai số GPS của người quan sát. Sai số 0.5m của laser bị lấn át. Mô hình RSS của 1 đã tính cỡ tính toán đúng độ lớn của trục lớn nhất của elip, nhưng nó đã hoàn toàn bỏ lỡ thông tin về hình dạng và hướng của vùng sai số.

## 10 Xem xét thuật toán tính toán tọa độ mục tiêu khác

### 10.1 Tổng quan

Đề tài tập trung xây dựng các thuật toán nhằm tính toán tọa độ địa lý của mục tiêu (kinh độ, vĩ độ, cao độ) dựa trên dữ liệu đầu vào gồm:

- Tọa độ GPS của người quan sát  $(\varphi_0, \lambda_0, h_0)$ .
- Góc ngắm *azimuth* và *elevation* thu được từ cảm biến IMU/la bàn.
- Khoảng cách đến mục tiêu  $d$  thu được từ cảm biến laser rangefinder.

Hai hướng tiếp cận chính được xem xét:

1. **Thuật toán Geometric (ECEF→ENU):** chuyển tọa độ người quan sát sang hệ tọa độ địa tâm (ECEF), tạo vectơ hướng trong hệ ENU, quay về ECEF, cộng vectơ dịch chuyển và chuyển ngược sang hệ tọa độ địa lý. Phương pháp này chính xác trên phạm vi toàn cầu. Đây là thuật toán nhóm lựa chọn, đã được thực hiện.
2. **Thuật toán Flat-Earth (cải tiến):** coi vùng xung quanh người quan sát là mặt phẳng tiếp tuyến với Trái Đất, chuyển đổi khoảng cách theo phương Đông - Bắc - Lên, sau đó quy đổi về độ vĩ và độ kinh. Phương pháp này nhanh, nhẹ, đủ chính xác cho phạm vi vài km.

### 10.2 Ký hiệu và hằng số

- Bán trục lớn  $a = 6,378,137.0$  m.
- Độ dẹt  $f = 1/298.257223563$ .
- Độ lệch tâm  $e^2 = f(2 - f)$ .
- Vĩ độ, kinh độ người quan sát:  $\varphi_0, \lambda_0$  (độ).
- Góc azimuth  $A$ , góc elevation  $el$  (độ).
- Khoảng cách nghiêng đến mục tiêu  $d$  (m).

### 10.3 Thuật toán Flat-Earth (cải tiến)

Giả sử các giá trị góc được chuyển sang radian:  $\alpha = A \cdot \pi/180, \beta = el \cdot \pi/180$ .

#### 10.3.1 Bước 1: Phân tách khoảng cách theo phương ngang và đứng

$$d_h = d \cos \beta, \quad U = d \sin \beta$$

#### 10.3.2 Bước 2: Tọa độ ENU của mục tiêu so với người quan sát

$$E = d_h \sin \alpha, \quad N = d_h \cos \alpha$$

### 10.3.3 Bước 3: Bán kính cong tại vĩ độ trung bình $\varphi_m$

Đây là bước quan trọng nhất của thuật toán. Tên gọi "Flat-Earth" (Trái Đất phẳng) là một tên gọi sai lầm. Một mô hình Trái Đất phẳng thực sự sẽ sử dụng định lý Pitago đơn giản và giả định một Trái Đất hình cầu.

Thuật toán này tinh vi hơn nhiều. Việc nó sử dụng các công thức cho  $M(\varphi_m)$  và  $N_p(\varphi_m)$  cho thấy nó không phải là mô hình Trái Đất phẳng, mà là một mô hình Xấp xỉ Ellipsoid Cục bộ (Local Ellipsoidal Approximation). Tên gọi "cải tiến" (improved) chính là để chỉ điều này.

Thuật toán này không giả định Trái Đất phẳng; nó giả định mặt phẳng tiếp tuyến tại vĩ độ trung bình  $\varphi_m$  là một xấp xỉ tốt và sử dụng các bán kính cong của ellipsoid tại điểm đó để tuyến tính hóa phép chuyển đổi từ mét sang độ.

$$M(\varphi_m) = \frac{a(1-e^2)}{(1-e^2 \sin^2 \varphi_m)^{3/2}},$$

$$N_p(\varphi_m) = \frac{a}{\sqrt{1-e^2 \sin^2 \varphi_m}}.$$

- Giải thích  $M(\varphi_m)$  (Bán kính Cong Kinh tuyến): Đây là bán kính của đường cong theo hướng Bắc-Nam (dọc theo một kinh tuyến). Công thức  $M(\varphi_m) = \frac{a(1-e^2)}{(1-e^2 \sin^2 \varphi_m)^{3/2}}$  là công thức trắc địa tiêu chuẩn cho bán kính này.<sup>14</sup> Nó được sử dụng để chuyển đổi khoảng cách  $N$  (Bắc).
- Giải thích  $N_p(\varphi_m)$  (Bán kính Cong Thăng đứng): Đây là bán kính của đường cong theo hướng Đông-Tây (vuông góc với kinh tuyến). Công thức  $N_p(\varphi_m) = \frac{a}{\sqrt{1-e^2 \sin^2 \varphi_m}}$  là công thức trắc địa tiêu chuẩn.<sup>14</sup> Nó được sử dụng như một phần của phép chuyển đổi khoảng cách  $E$  (Đông).

### 10.3.4 Bước 4: Quy đổi từ ENU sang thay đổi tọa độ địa lý

Bước này áp dụng công thức cơ bản của cung tròn (chiều dài cung = bán kính  $\times$  góc [rad]) để tìm sự thay đổi góc từ các khoảng cách  $N$  và  $E$ .

$$\Delta\varphi = \frac{N}{M},$$

$$\Delta\lambda = \frac{E}{N_p \cos \varphi_m}.$$

- Giải thích  $\Delta\varphi = N/M$ : Phép chuyển đổi vĩ độ rất đơn giản. Sự thay đổi góc vĩ độ (radian)  $\Delta\varphi$  là chiều dài cung (khoảng cách Bắc  $N$ ) chia cho bán kính của đường cong Bắc-Nam ( $M$ ).
- Giải thích  $\Delta\lambda = E/(N_p \cos \varphi_m)$ : Phép chuyển đổi kinh độ tinh vi hơn. Một sai lầm phổ biến là chia  $E$  cho  $N_p$ . Tuy nhiên, bán kính của một chuyển động Đông-Tây không phải là  $N_p$  (là bán kính thăng đứng). Bán kính của vòng tròn mà một người đi theo khi di chuyển về phía Đông là bán kính của vòng tròn vĩ tuyến (parallel of latitude) tại vĩ độ đó. Như 15 giải thích, các vòng tròn vĩ tuyến co lại từ xích đạo (nơi bán kính của chúng là  $N_p$ ) về các cực (nơi bán kính của chúng bằng 0), theo hàm cosin của vĩ độ. Do đó, bán kính của vòng tròn vĩ tuyến tại  $\varphi_m$  là  $N_p(\varphi_m) \cdot \cos(\varphi_m)$ . Vì vậy, sự thay đổi góc kinh độ (radian)  $\Delta\lambda$  là chiều dài cung (khoảng cách Đông  $E$ ) chia cho bán kính của vòng tròn vĩ tuyến ( $N_p \cos \varphi_m$ ). Thuật toán trong 1 đã thực hiện điều này một cách chính xác.

### 10.3.5 Bước 5: Tọa độ mục tiêu

$$\varphi_1 = \varphi_0 + \Delta\varphi \cdot \frac{180}{\pi},$$

$$\lambda_1 = \lambda_0 + \Delta\lambda \cdot \frac{180}{\pi},$$

$$h_1 = h_0 + U.$$

Các tọa độ ngang đơn giản là phép cộng đại số. Tuy nhiên, phép tính độ cao,  $h_1 = h_0 + U$ , là một sự đơn giản hóa quan trọng sẽ được phân tích trong phần tiếp theo. Phương pháp này cho độ chính xác rất cao trong phạm vi ngắn (sai số  $\approx$  mm với khoảng cách dưới 500 m), nhưng sai lệch tăng theo khoảng cách do giả thiết mặt phẳng tiếp tuyến.

## 10.4 Ví dụ minh họa (chi tiết) — Flat-Earth (improved)

Trong phần này ta làm một ví dụ số **từng bước** theo đúng các công thức của phương pháp Flat-Earth (improved).

### Dữ liệu đầu vào

$$\varphi_0 = 10.000000^\circ, \quad \lambda_0 = 106.000000^\circ, \quad h_0 = 10.0 \text{ m}, \\ \text{Azimuth } A = 63.232668^\circ, \quad \text{Elevation } el = 2.330536^\circ, \quad d = 245.799463 \text{ m}.$$

### Bước 1: Quy đổi góc sang radian

$$\alpha = A \cdot \frac{\pi}{180} = 63.232668^\circ \cdot \frac{\pi}{180} \approx 1.103997240 \text{ rad}, \\ \beta = el \cdot \frac{\pi}{180} = 2.330536^\circ \cdot \frac{\pi}{180} \approx 0.040673593 \text{ rad}.$$

### Bước 2: Tách khoảng cách thành thành phần ngang và đứng

$$d_h = d \cos \beta = 245.799463 \cdot \cos(0.040673593) \approx 245.5961536172 \text{ m}, \\ U = d \sin \beta = 245.799463 \cdot \sin(0.040673593) \approx 9.9952658558 \text{ m}.$$

### Bước 3: Tính vector ENU (East, North, Up)

$$E = d_h \sin \alpha \approx 245.5961536172 \cdot \sin(1.103997240) \approx 219.27874458780065 \text{ m}, \\ N = d_h \cos \alpha \approx 245.5961536172 \cdot \cos(1.103997240) \approx 110.60878285000351 \text{ m}, \\ U = 9.995265855755704 \text{ m} \quad (\text{như trên}).$$

### Bước 4: Tính bán kính cong tại vĩ độ trung bình

Vì khoảng cách nhỏ, ta chọn vĩ độ trung bình  $\varphi_m \approx \varphi_0$ . Dùng hằng số WGS-84:

$$a = 6378137.0 \text{ m}, \quad f = \frac{1}{298.257223563}, \quad e^2 = f(2 - f).$$

Tính:

$$\varphi_m = \varphi_0 \cdot \frac{\pi}{180} = 10^\circ \cdot \frac{\pi}{180} \approx 0.174532925199433 \text{ rad}, \\ M(\varphi_m) = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 \varphi_m)^{3/2}} \approx 6337358.121554948 \text{ m}, \\ N_p(\varphi_m) = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi_m}} \approx 6378780.843661353 \text{ m}.$$

### Bước 5: Tính $\Delta\varphi, \Delta\lambda$ (rad) và đổi sang độ

$$\Delta\varphi = \frac{N}{M} = \frac{110.60878285000351}{6337358.121554948} \approx 1.745345311539129 \times 10^{-5} \text{ rad}, \\ \Delta\lambda = \frac{E}{N_p \cos \varphi_m} = \frac{219.27874458780065}{6378780.843661353 \cdot \cos(0.174532925199433)} \\ \approx 3.490658765877035 \times 10^{-5} \text{ rad}.$$

Đổi về độ:

$$\Delta\varphi_{\text{deg}} = \Delta\varphi \cdot \frac{180}{\pi} \approx 0.0010000092014413793^\circ, \\ \Delta\lambda_{\text{deg}} = \Delta\lambda \cdot \frac{180}{\pi} \approx 0.0020000001500509864^\circ.$$



#### Bước 6: Tọa độ mục tiêu (theo Flat-Earth improved)

$$\varphi_1 = \varphi_0 + \Delta\varphi_{\text{deg}} = 10.000000^\circ + 0.0010000092014413793^\circ \\ \approx 10.001000009201441^\circ,$$

$$\lambda_1 = \lambda_0 + \Delta\lambda_{\text{deg}} = 106.000000^\circ + 0.0020000001500509864^\circ \\ \approx 106.00200000015005^\circ,$$

$$h_1 = h_0 + U = 10.0 + 9.995265855755704 \approx 19.995265855755704 \text{ m.}$$

### 10.5 Kết quả thử nghiệm và so sánh

Với bộ dữ liệu thử nghiệm:

$$\varphi_0 = 10.000000^\circ, \quad \lambda_0 = 106.000000^\circ, \quad h_0 = 10.0 \text{ m,} \\ A = 63.232668^\circ, \quad el = 2.330536^\circ, \quad d = 245.799463 \text{ m.}$$

Kết quả tính toán:

**Bảng 14:** So sánh kết quả giữa hai thuật toán

| Thuật toán            | $\varphi_1$ (deg)          | $\lambda_1$ (deg)           | $h_1$ (m)                |
|-----------------------|----------------------------|-----------------------------|--------------------------|
| Flat-Earth (cải tiến) | 10.0010000092              | 106.00200000015             | 19.9953                  |
| Geometric (ECEF)      | 10.0010000000              | 106.0020000000              | 20.0000                  |
| Sai lệch              | $\approx 0.001 \text{ mm}$ | $\approx 0.0001 \text{ mm}$ | $\approx 4.7 \text{ mm}$ |

Sai số giữa hai thuật toán là rất nhỏ (ở mức milimét), chứng tỏ phương pháp Flat-Earth hoàn toàn phù hợp cho phạm vi vài trăm mét.

### 10.6 Phân tích sai số và lựa chọn thuật toán

Phân tích Bảng 7 cho thấy một sự không tương xứng rõ rệt: chênh lệch ngang (vĩ độ, kinh độ) ở mức sub-milimét, trong khi chênh lệch dọc (cao độ) lớn hơn hàng nghìn lần, ở mức 4.7 mm.

Sự chênh lệch này không phải là ngẫu nhiên. Nó là kết quả trực tiếp của sự đơn giản hóa trong Bước 5 của thuật toán Flat-Earth.

- Như đã phân tích trong Phần 3, các bước tính toán ngang (E, N) của thuật toán Flat-Earth là các xấp xỉ ellipsoid cục bộ tinh vi (sử dụng  $M$  và  $N_p \cos \varphi_m$ ). Đây là lý do tại sao chúng gần như hoàn toàn khớp với kết quả của thuật toán Geometric (sai số 0.001 mm).
- Tuy nhiên, bước tính toán dọc là  $h_1 = h_0 + U$ .  $U$  là thành phần "Lên" (Up) so với mặt phẳng tiếp tuyến của người quan sát.
- Do độ cong của Trái Đất, vector "Up" (vuông góc với ellipsoid) tại vị trí mục tiêu (cách 245m) không song song với vector "Up" tại vị trí người quan sát.
- Thuật toán Geometric (ECEF) tự động tính toán điều này. Bằng cách giải  $(X_1, Y_1, Z_1)$  trong không gian 3D và sau đó giải  $h_1$  từ vị trí đó (Bước 2.5), nó tìm thấy độ cao của mục tiêu so với bề mặt ellipsoid bên dưới nó.
- Do đó, chênh lệch 4.7 mm chính là sai số hình học—độ "rơi" (drop) của đường chân trời—do việc bỏ qua độ cong của Trái Đất trong phép tính độ cao trên khoảng cách 245m. Mặc dù nhỏ, nó cho thấy điểm yếu lý thuyết của mô hình Flat-Earth: nó xử lý độ cao một cách "phẳng" trong khi xử lý vị trí ngang một cách "cong" (ellipsoidal).

Từ đó có thể đưa ra so sánh cơ bản:

- Thuật toán Flat-Earth (cải tiến): Như được trình bày trong , toàn bộ thuật toán bao gồm một số lượng cố định các phép toán đại số và lượng giác (sin, cos, sqrt, pow). Không có vòng lặp. Chi phí tính toán của nó thấp và, quan trọng hơn, có thể dự đoán được (deterministic).
- Thuật toán Geometric (ECEF ENU): Thuật toán này bao gồm tất cả các phép tính của thuật toán Flat-Earth (trong các bước LLA  $\rightarrow$  ECEF và AER  $\rightarrow$  ENU) cộng thêm điểm nhên tính toán (ECEF  $\rightarrow$  LLA). Bước ECEF  $\rightarrow$  LLA đòi hỏi một giải pháp lặp (như Newton-Raphson) hoặc giải một phương trình bậc 4 phức tạp. Do đó, thuật toán Flat-Earth "nhanh hơn" không chỉ vì nó có ít phép toán hơn, mà vì nó tránh được bài

toán nghịch đảo (inverse problem) vốn đòi hỏi phép lặp. Phép cộng đại số đơn giản trong Bước 5 của nó ( $\varphi_1 = \varphi_0 + \Delta\varphi$ ) thực chất là một xấp xỉ tuyến tính, một bước của bài toán nghịch đảo, thay thế một vòng lặp có chi phí thay đổi bằng một phép toán có chi phí cố định và thấp.

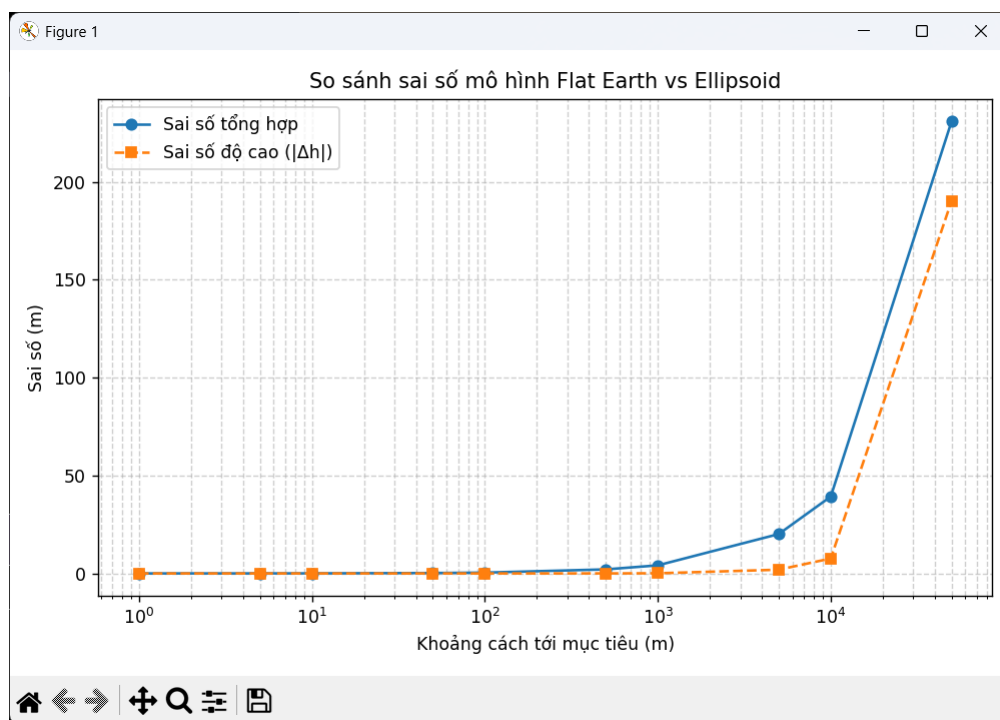
Vậy, có thể thấy:

- **Nguồn sai số:** do giả thiết mặt phẳng, độ cong ellipsoid bị bỏ qua, sai số cảm biến (GPS, la bàn, IMU, laser).
- **Khoảng cách ngắn (< 1 km):** sai số của Flat-Earth nhỏ hơn 0.1 m.
- **Khoảng cách trung bình (1–5 km):** sai số tăng lên vài mét, có thể chấp nhận cho UAV hoặc hệ quan sát tầm gần.
- **Khoảng cách lớn (> 10 km):** cần dùng Geometric để đảm bảo chính xác.

## 10.7 Đánh giá và biểu diễn kết quả

Nhóm tiến hành so sánh theo các tiêu chí:

- **Sai số vị trí (m)** giữa hai thuật toán.
- **Ảnh hưởng của nhiễu** trong các góc đo và khoảng cách.
- **Hiệu năng tính toán** (thời gian xử lý).



Hình 6: Đồ thị biểu diễn kết quả so sánh sai số giữa 2 mô hình

## 10.8 Kết luận

- Thuật toán **Flat-Earth (cải tiến)** đạt độ chính xác tương đương Geometric trong phạm vi vài km, đồng thời tính toán nhanh và đơn giản, phù hợp cho hệ thống nhúng hoặc UAV.
- Thuật toán **Geometric (ECEF→ENU)** đảm bảo tính chính xác toàn cầu, thích hợp cho ứng dụng đo xa hoặc yêu cầu sai số ở mức cm.

## 11 Tổng kết và Hướng phát triển

### 11.1 Tổng kết Giai đoạn 1

Trong Giai đoạn 1 của đề án, nhóm đã tập trung nghiên cứu cơ sở lý thuyết và xây dựng nền tảng ban đầu cho hệ thống tính toán tọa độ mục tiêu. Các kết quả đạt được bao gồm:

- **Nghiên cứu lý thuyết:**
  - Tìm hiểu sâu về hệ thống định vị toàn cầu GPS, các hệ tọa độ không gian (Geodetic, ECEF, ENU, NED) và các phép chuyển đổi giữa chúng.
  - Phân tích các nguồn sai số trong GPS và cảm biến (IMU, Laser Rangefinder) cũng như các phương pháp hiệu chuẩn cơ bản.
  - Nắm vững nguyên lý tính toán trắc địa trên mặt cầu (Haversine) và mặt ellipsoid (Vincenty).
- **Hiện thực ứng dụng (Prototype):**
  - Xây dựng thành công ứng dụng Web Map Viewer sử dụng thư viện Leaflet.js.
  - Hiện thực module tính toán tọa độ mục tiêu từ vị trí quan sát, góc phương vị và khoảng cách với giao diện trực quan.
  - Tích hợp các công cụ hỗ trợ như chuyển đổi định dạng tọa độ (DMS/Decimal), đo đạc khoảng cách và hiển thị trực quan trên bản đồ số.

Hệ thống hiện tại hoạt động ổn định trên trình duyệt, đáp ứng tốt các yêu cầu cơ bản về tính toán và hiển thị trong phạm vi khoảng cách ngắn (< 100km) với sai số chấp nhận được cho các bài toán quan sát thông thường.

### 11.2 Hạn chế

Mặc dù đã đạt được những kết quả bước đầu, hệ thống vẫn còn một số hạn chế cần khắc phục:

- **Mô hình toán học:** Vẫn sử dụng công thức Haversine trên mô hình cầu, chưa áp dụng triệt để mô hình Ellipsoid WGS-84 cho độ chính xác cao nhất ở khoảng cách xa.
- **Dữ liệu độ cao:** Chưa tích hợp dữ liệu độ cao (Elevation/DEM), dẫn đến việc tính toán chỉ dừng lại ở mặt phẳng 2D, bỏ qua sai số do chênh lệch độ cao giữa người quan sát và mục tiêu.
- **Lưu trữ:** Chưa có Backend và Cơ sở dữ liệu để lưu trữ lịch sử đo đạc, quản lý người dùng và chia sẻ dữ liệu giữa các nhóm.

### 11.3 Hướng phát triển Giai đoạn 2

Dựa trên kết quả và hạn chế của Giai đoạn 1, nhóm đề xuất kế hoạch phát triển cho Giai đoạn 2 như sau:

#### 1. Nâng cấp thuật toán:

- Chuyển đổi hoàn toàn sang công thức Vincenty hoặc Karney để tính toán trên Ellipsoid WGS-84.
- Tích hợp API lấy dữ liệu độ cao (Google Elevation API hoặc OpenTopography) để tính toán 3D (Slant range to Horizontal distance).

#### 2. Phát triển Backend:

- Xây dựng Server (Node.js/Python) và Database (PostgreSQL/PostGIS) để quản lý dữ liệu không gian.
- Hiện thực API RESTful cho phép ứng dụng mobile hoặc thiết bị nhúng gửi dữ liệu trực tiếp.

#### 3. Tích hợp phần cứng (Hardware Integration):

- Kết nối thử nghiệm với module GPS (u-blox), IMU (MPU6050/9250) và Laser Rangefinder thực tế.
- Xây dựng luồng dữ liệu từ cảm biến → Vi điều khiển → Web Server.

#### 4. Nâng cao trải nghiệm người dùng:

- Hỗ trợ bản đồ Offline (MBTiles).
- Thêm tính năng dẫn đường (Routing) đến mục tiêu đã xác định.

## Tài liệu tham khảo

- [1] Vladimir Agafonkin. *Leaflet - an open-source JavaScript library for mobile-friendly interactive maps*, 2023. Accessed: 2023-11-26.
- [2] Markus-Christian Amann, Thierry Bosch, Marc Lescure, Risto Myllyla, and Marc Rioux. *Laser ranging: a critical review of usual techniques for distance measurement*, volume 40. SPIE, 2001.
- [3] Jon Duckett. *JavaScript and JQuery: Interactive Front-End Web Development*. John Wiley & Sons, Indianapolis, IN, 2014.
- [4] Jay A Farrell. Aided navigation: Gps with high rate sensors. *McGraw-Hill Professional*, 2008.
- [5] David Flanagan. *JavaScript: The Definitive Guide*. O'Reilly Media, Sebastopol, CA, 7th edition, 2020.
- [6] Charles D Ghilani. *Adjustment computations: Spatial data analysis*. 2010.
- [7] Paul D Groves. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Artech House, Boston, MA, 2nd edition, 2013.
- [8] Bernhard Hofmann-Wellenhof, Herbert Lichtenegger, and James Collins. *Global Positioning System: Theory and Practice*. Springer Science & Business Media, Vienna, Austria, 5th edition, 2012.
- [9] Elliott D Kaplan and Christopher J Hegarty. *Understanding GPS/GNSS: Principles and Applications*. Artech House, Boston, MA, 3rd edition, 2017.
- [10] Charles FF Karney. Algorithms for geodesics. *Journal of Geodesy*, 87(1):43–55, 2013.
- [11] Paul A Longley, Michael F Goodchild, David J Maguire, and David W Rhind. *Geographic Information Science and Systems*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2015.
- [12] Pratap Misra and Per Enge. *Global Positioning System: Signals, Measurements, and Performance*. Ganga-Jamuna Press, Lincoln, MA, 2nd edition, 2011.
- [13] National Imagery and Mapping Agency. *Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems*. National Imagery and Mapping Agency, Bethesda, MD, 2000. NIMA TR8350.2.
- [14] National Marine Electronics Association, Severna Park, MD. *NMEA 0183 Standard for Interfacing Marine Electronic Devices*, version 4.10 edition, 2002.
- [15] Gretchen N Peterson. *GIS Cartography: A Guide to Effective Map Design*. CRC Press, Boca Raton, FL, 2nd edition, 2014.
- [16] Dan Simon. *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. John Wiley & Sons, Hoboken, NJ, 2006.
- [17] Roger W Sinnot. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.
- [18] John P Snyder. *Map Projections: A Working Manual*. Number 1395 in US Geological Survey Professional Paper. US Government Printing Office, Washington, DC, 1987.
- [19] John R Taylor. *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. University Science Books, Sausalito, CA, 2nd edition, 1997.
- [20] David Titterton and John L Weston. *Strapdown Inertial Navigation Technology*. IET, London, UK, 2nd edition, 2004.
- [21] Thaddeus Vincenty. Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations. *Survey Review*, 23(176):88–93, 1975.
- [22] Greg Welch and Gary Bishop. An introduction to the kalman filter. *University of North Carolina at Chapel Hill, Chapel Hill, NC*, 7(1):1–16, 1995.
- [23] Oliver J Woodman. An introduction to inertial navigation. *University of Cambridge, Computer Laboratory, Tech. Rep. UCAMCL-TR-696*, 14:15, 2007.